

Arculus Recovery

User & Developer Manual

Version 1.5.0

Published June 2026

Covers	HTML application · Python CLI/GUI · Cryptographic internals · Recovery procedures · Operational security
Audience	Wallet owners recovering funds, developers auditing or reimplementing the engine, security reviewers
Files	Arculus_Recovery.html (browser, air-gapped) · Arculus_Recovery.py (CLI/GUI, air-gapped)
Handling	Any session involving real mnemonics must follow the air-gap procedures in Chapter 5

■ This document contains detailed instructions for working with cryptographic seed material. Any file or printout derived from a live mnemonic must be stored with the same physical security as the seed phrase itself. Do not photograph this document with a networked device.

Contents

Ch.	Title	About
1	Introduction	What Arculus Recovery is, how to get started
2	Quick Start	Enter your mnemonic and derive your first addresses
3	The Interface	Every control explained, from word grid to export panel
4	Recovery Step-by-Step	Detailed procedures for every recovery scenario
5	Operational Security	Air-gap setup, session hygiene, storage, threat model
6	The BIP39 Passphrase	What it is, why it matters, how to back it up safely
7	Encrypted Seed Files (.arc)	Creating, importing, and verifying .arc backups
8	QR Code Export	Generating receive QR codes, XRP destination tags
9	Derivation Engine	BIP39, PBKDF2, BIP32, address construction internals
10	Encryption Internals	HMAC-SHA512-CTR stream cipher, key schedule, MAC
11	File Format Reference	Schemas for .arc, JSON, CSV, TXT, and PDF exports
12	Test Vectors	Canonical vectors for validating reimplementations
13	Source Code	Complete annotated HTML source listing

Chapter 1

What is Arculus Recovery?

Arculus Recovery is a self-contained, offline tool for deriving cryptocurrency addresses and private keys from a BIP39 seed phrase. It runs entirely in your browser or as a Python script with no internet connection required — and by design, no internet connection is ever made. There are no accounts, no servers, and no third-party services involved at any point.

The tool comes in two forms that are functionally identical. The HTML version opens in any modern browser directly from a local file. The Python version runs from the command line or as a desktop GUI. Both produce byte-for-byte identical results for the same inputs, so you can verify one implementation against the other.

What it can do

- Validate a 12- or 24-word BIP39 mnemonic, including the checksum
- Generate a cryptographically random mnemonic using the platform's secure random number generator
- Derive addresses and private keys for Bitcoin (all script types), Litecoin, Dogecoin, Ethereum, and XRP
- Export derived keys as JSON, CSV, plain text, or PDF
- Generate receive QR codes for any supported address format
- Save your seed phrase to an encrypted .arc backup file, and import it back
- Run entirely offline on an air-gapped machine with no dependencies beyond a browser or Python

What it cannot do

Arculus Recovery is a key derivation and inspection tool, not a wallet. It never queries any blockchain, checks balances, broadcasts transactions, or connects to any network. If you need to know whether an address holds funds, you'll need to look that up separately on a blockchain explorer after your offline session is complete.

Supported coins and address types

Coin	Address Formats	Default Path
Bitcoin	P2PKH (1...), P2WPKH-P2SH (3...), P2WPKH (bc1q...), P2TR (Arclius) or m/84'/0'/0'	m/84'/0'/0'
Litecoin	P2PKH (L...), P2WPKH-P2SH (M...), P2WPKH (ltc1q...), P2TR (Arclius) or m/84'/0'/0'	m/84'/0'/0'
Dogecoin	P2PKH (D...)	m/44'/3'/0'
Ethereum	EIP-55 checksummed (0x...)	m/44'/60'/0'
XRP	Classic r-address	m/44'/144'/0'

System requirements

For the HTML version, any modern browser with JavaScript enabled works — Chrome, Firefox, Edge, or Safari. No installation needed; just open the file. For the Python version, Python 3.9 or later is required. The script uses only standard library modules (hashlib, hmac, json, base64, unicodedata, and a few others), so there is nothing to install beyond Python itself. Tkinter is only required if you want the desktop GUI mode (`--gui` flag).

Chapter 2

■ Before working with a real mnemonic, read Chapter 5 (Operational Security). The five-minute setup described here assumes you are on an air-gapped machine.

HTML: open the file in your browser

1. Copy Arculus_Recovery.html to a USB drive or directly onto the target machine.
2. Disconnect the machine from all networks (unplug Ethernet, disable Wi-Fi at the hardware level).
3. Open the HTML file in your browser. Look at the top-right corner of the page — the network indicator should show red (offline). If it shows green, stop and re-check your network connections.
4. You are ready to begin.

Python: run from the command line

```
# Derive the first 5 Bitcoin native-SegWit receiving addresses
python Arculus_Recovery.py \
  --mnemonic "word1 word2 ... word12" \
  --coin bitcoin \
  --derivation "m/84'/0'/0'" \
  --script-type p2wpkh \
  --count 5 \
  --output-format txt
```

Enter your mnemonic

In the HTML interface, you will see a numbered grid of word fields. Select either 12 or 24 words using the radio buttons at the top, then type each word into its numbered box. As you type, each field turns green when the word is recognized as a valid BIP39 word, and red if it is not. You can also paste your entire mnemonic into any cell and the tool will distribute the words automatically across all the fields.

Once all words are entered, the tool validates the BIP39 checksum. If the checksum fails, at least one word is wrong — either misspelled or in the wrong position. The tool tells you which check failed. Correct any flagged words before proceeding.

■ A checksum failure is a clear signal that something is wrong. A passing checksum is necessary but not sufficient — it is still possible to have a plausible-looking mnemonic that passes the checksum but does not match your wallet. Always verify the root fingerprint against a previously recorded value.

Derive addresses

Select your coin, choose a derivation path and script type (or leave them at their defaults if you know the wallet you are recovering from), set the address count, and click Derive Keys + Addresses. The output panel fills in a few seconds with receiving and change addresses, their public keys, and their private keys.

The root fingerprint — an 8-character hex string shown in the title bar — updates as soon as the mnemonic is valid. This fingerprint is your primary verification checkpoint: if it matches the fingerprint you recorded when the

wallet was first created, you have the right mnemonic and passphrase.

Export your results

Use the format selector at the bottom of the output panel to choose JSON, CSV, TXT, or PDF, then click Export. Save the file to an encrypted USB drive — not to the machine's built-in storage. JSON is the most complete format (preserving all key fields including extended keys). PDF is the most readable for printing. TXT works well for a quick visual check.

Chapter 3

The title bar

At the top of the page you will find the title, a root fingerprint display, and a small network status indicator. The network indicator polls your system every few seconds. Green means a network route exists — stop and disconnect before doing anything. Red means the machine is offline, which is what you want.

The root fingerprint appears as an 8-character hex value next to the key icon. It updates automatically whenever the mnemonic or passphrase changes. If you have a record of this value from a previous session, compare it here before trusting any derived output.

Word entry grid

The grid always shows 24 numbered word fields. When you select 12-word mode, the last 12 fields are disabled and shown greyed out. Each field validates your input against the 2,048-word BIP39 English word list as you type — green border for a valid word, red for an unrecognized word.

Pasting: if you paste a full mnemonic into any word field, the tool detects the spaces, splits the string into individual words, and distributes them starting from field 1. This works regardless of which field you paste into.

Hidden seed mode: after importing a .arc file, the word fields show placeholder bullets rather than your actual words. The mnemonic is held in protected memory but not displayed unless you press and hold the Show Seed button. Release the button and the words hide again.

Passphrase field

The optional BIP39 passphrase goes here. By default the field is masked (contents shown as dots). Click the eye icon on the right edge to reveal what you have typed. Use this sparingly and only to verify correctness — not while anyone can see your screen.

■ The passphrase field and the .arc encryption password are two completely different things. The passphrase is part of your wallet's key derivation. The .arc password only protects the backup file. They can be the same string, but they serve different purposes and are processed by entirely different cryptographic pipelines. See Chapter 6 for more.

Coin, path, and script type

The coin selector sets the target blockchain. For Bitcoin specifically, you also choose a script type — this determines the address format (legacy 1..., wrapped SegWit 3..., native SegWit bc1q..., or Taproot bc1p...). The Auto setting infers the script type from the purpose field of the derivation path, which is usually the right choice.

The derivation path field accepts standard BIP32 path notation. Hardened components can be marked with an apostrophe (84'), a lowercase h (84h), or an uppercase H (84H) — all three are equivalent. The field always displays in apostrophe form.

If you are not sure which path your wallet used, see the path reference table in Chapter 4.

Address count and range

The Count field sets how many addresses to derive per branch (receiving and change). The default is 5. If your target address is not in the first 5, increase this number or use the Range fields to start from a specific index. For example, setting start=20 and count=10 derives addresses at indices 20 through 29.

Generate Random Seed

This button creates a fresh cryptographically random mnemonic using the browser's secure random number generator. The generated mnemonic is immediately validated against its own BIP39 checksum before being accepted. Use this to create a new wallet seed — then export it as a .arc backup immediately.

■ A randomly generated mnemonic is only as safe as the machine it was generated on. Generate seeds exclusively on air-gapped machines running from a live OS that writes nothing to disk.

The output panel

After derivation, the output panel shows your results in one of two views. Table View lists each address with its branch (receiving or change), path, address string, public key, private key hex, and WIF private key. Each row has a small QR button that opens the QR export modal for that address. Detail View shows the same information in a more readable vertical layout, along with account-level extended keys.

Export controls

At the bottom of the output panel is a format selector (JSON / CSV / TXT / PDF) and an Export button. JSON is the most complete, preserving the full nested structure including extended keys at every level. CSV is useful if you want to open the results in a spreadsheet. TXT is human-readable for a quick visual check. PDF is formatted for printing.

■ Every export format except plain addresses contains private key material. Treat any exported file with the same security as your seed phrase — encrypted storage, offline transfer only.

Import Seed

The Import Seed button opens a file picker for loading a .arc encrypted seed file. After you select the file, you are prompted for the decryption password. The tool verifies the file's authentication tag before decrypting — a wrong password or a tampered file both produce the same authentication error message, by design.

Export Seed (.arc)

This saves the current mnemonic as an encrypted .arc backup. You are prompted to enter and confirm a password. The resulting file is safe to store on a USB drive; the mnemonic cannot be recovered without the password. Chapter 7 covers the format and cryptography in detail.

Clear All

Wipes all fields — mnemonic, passphrase, derivation output, and any imported seed state. Use this at the end of every session before closing the tab. On an ephemeral live OS, powering off the machine is a stronger guarantee than clicking Clear All, because power-off destroys RAM contents completely.

Chapter 4

Before you start: what you need

A complete BIP39 recovery requires up to four pieces of information. You may not need all of them, but you should gather what you have before starting a session.

Mnemonic	Your 12- or 24-word seed phrase. Every word must be spelled correctly and in the right order. A single wrong word usually fails the checksum, but not always.
Passphrase	Only if your wallet used one. Most wallets default to no passphrase. If you are not sure, try empty first.
Derivation path	The BIP32 path your wallet used. If you know the wallet software, look it up in the table below.
Script type	For Bitcoin: which address format (1..., 3..., bc1q..., or bc1p...). Other coins have only one format.

Path reference: what path did your wallet use?

Wallet / Setup	Coin	Path	Address format
Arculus hardware wallet	Bitcoin	m/0'	bc1q... (native SegWit)
Most modern wallets	Bitcoin	m/84'/0'/0'	bc1q... (native SegWit)
Legacy BIP44 wallets	Bitcoin	m/44'/0'/0'	1... (P2PKH)
Wrapped SegWit wallets	Bitcoin	m/49'/0'/0'	3... (P2SH)
Taproot wallets	Bitcoin	m/86'/0'/0'	bc1p... (Taproot)
Standard	Litecoin	m/84'/2'/0'	ltc1q...
Standard	Dogecoin	m/44'/3'/0'	D...
MetaMask / standard	Ethereum	m/44'/60'/0'	0x...
Standard	XRP	m/44'/144'/0'	r...

Procedure A: You know the wallet software

1. Open the tool on an air-gapped machine. Confirm the network indicator is red.
2. Enter your mnemonic word-by-word and confirm the BIP39 checksum passes.
3. Enter your BIP39 passphrase if one was used. If you are not sure, leave it blank and compare the root fingerprint to a known value.
4. Select the coin and enter the derivation path from the table above.
5. Set the script type to match the address format of the wallet (or leave Auto if the path uses a standard purpose field).

6. Click Derive Keys + Addresses.
7. Compare the root fingerprint to any previously recorded value.
8. Scroll through the receiving-branch addresses and confirm at least one matches an address you recognize from your wallet's history.
9. Export the results to an encrypted USB drive.

Procedure B: You don't know the path or script type

Start with the address format. Look at an address you know was in the wallet — the first character tells you a lot. For Bitcoin: 1 means P2PKH, 3 means P2SH, bc1q means native SegWit, bc1p means Taproot. Then try paths in this order, checking whether your known address appears in the first 10 receiving-branch results:

1. m/84'/0'/0' with P2WPKH — the most common modern Bitcoin path
2. m/0' with P2WPKH — the Arculus hardware wallet default
3. m/44'/0'/0' with P2PKH — legacy wallets
4. m/49'/0'/0' with P2WPKH-P2SH — wrapped SegWit
5. m/86'/0'/0' with P2TR — Taproot

If none of the first 10 addresses match, increase the count to 25, then 50. Also check the change branch (branch index 1). If still nothing, try account index 1 (e.g. m/84'/0'/1') — some wallets create multiple accounts.

The Python CLI's `--all-common` flag does all four Bitcoin purpose paths in one go, which saves time when searching:

```
python Arculus_Recovery.py \  
  --mnemonic "word1 word2 ... word12" \  
  --coin bitcoin \  
  --all-common \  
  --count 10 \  
  --output-format txt
```

Procedure C: Recovery from a .arc file

1. Transfer the .arc file to the air-gapped machine via USB drive.
2. Click Import Seed and select the file.
3. Enter the .arc decryption password (not the BIP39 passphrase — these are different).
4. If decryption succeeds, the mnemonic is loaded into protected memory. The root fingerprint updates immediately.
5. Verify the root fingerprint against your records.
6. Continue with coin selection and derivation as normal.

■ If you get an authentication error, check that you typed the password correctly. Both a wrong password and a corrupted file produce the same error message — this is intentional, to avoid leaking information to an attacker. If the correct password definitely fails, the file may be damaged. Recover from a second copy or from your paper mnemonic backup.

Verifying your results

Before relying on any derived output, run through this checklist:

- The BIP39 checksum passed without errors.

- The root fingerprint matches a previously recorded value (or you have recorded it now for future sessions).
- At least one derived address matches an address you recognize from your wallet's history.
- The passphrase field contained exactly what you intended — empty if no passphrase was used.
- The derivation path and script type match what the originating wallet used.

Chapter 5

Why this matters

The cryptographic algorithms in Arculus Recovery are sound. What this cannot protect you against is a compromised machine, a screen that can be photographed, or a clipboard that syncs to the cloud. Operational security is the set of physical and procedural practices that make the math meaningful. None of them are complicated. All of them are necessary.

The air gap

Every session involving a real mnemonic must be conducted on a machine that has no active network connection. This means:

- Ethernet cables physically unplugged — not just disabled in software.
- Wi-Fi disabled at the hardware level — use the physical kill switch if available, or remove the card.
- Bluetooth disabled at the hardware level for the same reasons.
- The network status indicator in the tool shows red before you type a single word.

The network indicator is a convenience check. Physical confirmation of cable removal and switch state is authoritative. Do not rely on the indicator alone.

Use a live operating system

A machine that boots from a live USB drive (Tails or a standard Linux live environment) writes nothing to its internal disk. Every file you open, every key you type, and every address you see exists only in RAM. When you power off the machine, that RAM is erased completely. This is the only way to have high confidence that no trace of the mnemonic survives the session.

For the HTML version: open the local file from the live environment. Verify that the browser is not configured to sync anything to any account. For the Python version: copy the script to /tmp (which is RAM-backed on live Linux environments) before running.

If a dedicated offline machine is the only option, the swap partition and any temp directories should be explicitly wiped after the session. On SSDs, software wiping cannot reliably reach all storage cells due to wear-leveling — this is an inherent SSD limitation, not something the tool can address.

Physical environment

Close the room. No cameras, phones, or other screens should have line of sight to your display. This sounds obvious but it is the most frequently skipped precaution. A photo of your screen showing the mnemonic word grid or the private key output is as damaging as emailing the seed phrase to an attacker.

Session procedure

1. Prepare the physical environment and confirm privacy.
2. Boot from a live USB. Confirm the air gap (physical confirmation, then tool indicator).

3. Transfer the .arc file or the tool to the machine via a dedicated USB drive.
4. Open the tool and run the session.
5. Export results to the same USB drive (encrypted).
6. Click Clear All, then close the browser or terminal.
7. Power off the machine completely — do not suspend or hibernate.
8. Store the USB drive in a physically secured location.

Password requirements for .arc files

The .arc format uses 1,000,000 PBKDF2 iterations, which makes brute-force expensive. But no iteration count saves a weak password. Here is practical guidance:

Not acceptable	Any single dictionary word. Any PIN. Your name, birthday, or address. Any password reused from another account.
Marginal	Four randomly selected words from a large wordlist (Diceware). About 51 bits of entropy — enough to deter casual attackers, not determined ones.
Recommended	Six or more randomly selected Diceware words (~77 bits), or a 20+ character random alphanumeric string. This is comfortable against any foreseeable attack.

■ A randomly generated passphrase that you write down and store securely is far stronger than a clever string you invented. Human-composed passwords are systematically weaker than they feel. Use a dice or a CSPRNG, not your imagination.

Storing backups safely

- .arc files belong on encrypted, dedicated USB drives — not on your daily machine or cloud storage.
- The .arc password and the mnemonic paper backup must be stored in separate physical locations.
- A single burglary, fire, or flood should not be able to reach both the .arc file and its password.
- Verify the .arc round-trip (export then import, confirm fingerprint matches) before relying on it as your only backup.
- USB drives have a data retention of roughly 10 years under normal conditions. Re-export annually and store on at least two drives.

Chapter 6

What the passphrase is

The BIP39 passphrase is an optional string that you enter alongside your mnemonic. It is sometimes called the '25th word,' though it is not restricted to the BIP39 word list — it can be any text of any length, including the empty string (which means no passphrase).

The passphrase changes the seed that gets derived from your mnemonic. A mnemonic with an empty passphrase and the same mnemonic with the passphrase 'hello' produce completely different key trees, with different addresses and different private keys. There is no mathematical relationship between them.

Why it is the most dangerous part

When you enter a wrong mnemonic word, the checksum usually fails and you get an error. When you enter a wrong passphrase, you get no error at all. The tool derives a perfectly valid-looking set of addresses — they just belong to a wallet with no funds in it. The only way to detect a wrong passphrase is to compare the derived root fingerprint against a previously recorded value, or to recognize an address from your wallet's history.

■ Record the root fingerprint every time you set up a new passphrase. This 8-character hex string is not a secret — write it down alongside your mnemonic backup and your passphrase backup. It is the only reliable way to confirm that a passphrase was entered correctly during recovery.

What it is not

- It is not the .arc encryption password. The .arc password protects your backup file. The BIP39 passphrase participates in key derivation. They are processed by completely separate cryptographic pipelines.
- It is not stored in the .arc file. Decrypting a .arc file recovers the mnemonic only. The passphrase must be remembered or retrieved from a separate backup.
- It is not validated by any checksum. A typo in the passphrase produces no error.
- It is not recoverable from the mnemonic. There is no mathematical way to find the passphrase from the mnemonic, even if you know the derived addresses.

Case sensitivity and character handling

The passphrase is case-sensitive. 'Password' and 'password' are completely different. Trailing spaces matter. A space at the end produces a different wallet than the same string without the space. The only normalization applied is Unicode NFKD decomposition, which resolves composed versus decomposed characters (like accented letters) to the same byte sequence. For ASCII-only passphrases — the strongly recommended approach — this normalization has no effect at all.

Backup requirements

If your wallet uses a passphrase, it must be backed up independently of the mnemonic, and stored in a different physical location. A mnemonic backup without the passphrase can only recover the empty-passphrase wallet. If

your funds are in a passphrase-derived wallet and you lose the passphrase, those funds are gone permanently — including to the developers of this software.

The minimum acceptable backup setup for a wallet with a passphrase:

1. Mnemonic words written on paper or engraved on metal, stored in a fire-resistant container.
2. Passphrase written separately, stored in a different physical location from the mnemonic.
3. Root fingerprint recorded and stored in both locations (it is not a secret).
4. Optionally, a .arc encrypted backup of the mnemonic on USB, with its own password stored separately.

Verify all backups immediately after creating them. Enter the mnemonic and passphrase, confirm the root fingerprint, and confirm that at least one known address appears in the derivation output. Do not trust an unverified backup.

Plausible deniability

One common use of the passphrase is to maintain multiple separate wallets from the same mnemonic — typically a small decoy wallet with no passphrase, and the real wallet protected by a passphrase. An attacker who obtains the mnemonic cannot determine how many passphrases exist or which one contains the actual funds. This provides deniability under coercion.

If this is your setup, be aware that the 2,048 PBKDF2 iterations used by BIP39 for the passphrase are much lower than the 1,000,000 used for .arc files. A short, low-entropy passphrase provides weak resistance to offline brute-force if an attacker has your mnemonic. For meaningful hardening, the passphrase needs to be long and random — see the entropy guidance in Chapter 5.

Chapter 7

What a .arc file is

A .arc file is an encrypted backup of a single BIP39 mnemonic. It is a plain text file — you can open it in any text editor and see one line of header text and one line of base64 data. Inside that base64 envelope is a JSON structure holding the encrypted mnemonic, the salt and nonce used for encryption, and an authentication tag covering all of those fields.

The current format (ARCULUS-ARC-V2) uses PBKDF2-HMAC-SHA512 with 1,000,000 iterations to derive the encryption key from your password, and a custom HMAC-SHA512 stream cipher for the encryption itself. The authentication tag covers not just the ciphertext but every parameter field — KDF algorithm, iteration count, nonce, format version — so that any tampering is detected before decryption is even attempted.

What it protects and what it does not

Protected	The mnemonic phrase at rest, against an attacker who obtains the file without the password. All metadata integrity — any modification to any field is detected.
Not protected	The BIP39 passphrase (not stored in the file). Derived private keys (never written to .arc). The mnemonic while loaded in memory during a session. The password while you are typing it.

Creating a .arc backup

1. Enter or import your mnemonic and verify the root fingerprint.
2. Click Export Seed (.arc).
3. Type a strong password (see Chapter 5 for entropy requirements) and confirm it.
4. The file saves with a name like `arculus_seed_20260605_143200.arc`. You can rename it freely.
5. Immediately import the file back using Import Seed and the same password to verify the round-trip.
6. Confirm the root fingerprint after import matches what you recorded.
7. Copy the verified file to an encrypted USB drive.

■ Never rely on a .arc file as your backup until you have completed the export-import-verify cycle. A file written to failing storage media may be silently corrupted.

Importing a .arc file

1. Transfer the .arc file to the air-gapped machine via USB.
2. Click Import Seed and select the file.
3. Enter the decryption password when prompted.
4. If authentication fails, double-check the password. Both a wrong password and a corrupted file produce the same error.
5. On success, the mnemonic loads into protected memory. Verify the root fingerprint.

6. Continue with normal derivation.

Legacy format support

Arculus Recovery can import .arc files created by older versions of the tool, including formats that used AES-GCM (HTML only), PBKDF2-SHA256 with a simpler authentication scheme, and very early formats with no format field at all. When you import a legacy file and re-export, the output is always a fresh ARCULUS-ARC-V2 file with the current 1,000,000 iteration count. Re-exporting old files is recommended to upgrade them to current standards.

Maintenance

- Verify your .arc backup at least once a year: import it, confirm the fingerprint, and re-export.
- Store at least two copies on separate USB drives in separate physical locations.
- Never store the .arc file and its password in the same location.
- The .arc password and the BIP39 passphrase are separate secrets — both must survive independently for recovery to succeed.

Chapter 8

Overview

The QR Export subsystem lets you encode any address as a QR code, which you can then scan with a mobile wallet to initiate a receive transaction. The QR encoder is fully self-contained — no library is loaded from the internet, no data leaves the machine. It supports QR versions 1 through 10 and selects the smallest version that fits your address.

QR codes are generated in black on white, regardless of your theme setting. This is intentional: camera-based QR decoders in wallet apps are calibrated for high-contrast black-on-white patterns, and colored or dark backgrounds cause silent scan failures on many devices.

Opening the QR modal

There are two ways to open the QR modal. The QR Export button in the toolbar opens it blank — type or paste any address you want to encode. The small QR button (a grid icon) next to each row in Table View opens the modal with that address already filled in.

URI schemes

The tool automatically wraps addresses in the appropriate payment URI scheme before encoding. This produces the format that wallet apps expect:

Bitcoin	bitcoin:1abc... or bitcoin:bc1q...
Litecoin	litecoin:ltc1q...
Dogecoin	dogecoin:D...
Ethereum	ethereum:0x...
XRP	Bare address (r...) or r...?dt=TAGNUMBER

XRP destination tags

XRP deposits to exchange accounts (Coinbase, Binance, Robinhood, and others) typically require a destination tag — a number that routes the funds to your specific account within the exchange's shared deposit address. Sending without the required tag results in funds arriving at the exchange but not being credited to your account.

Because of this, the QR modal uses a two-step workflow for XRP addresses. After entering an XRP address and clicking Generate, the tool asks whether a destination tag is needed. If yes, enter the tag number the exchange provided and click Generate QR. If no tag is needed (sending to your own XRP wallet), click Skip — No Tag. The QR is then generated and the full URI shown below it so you can verify the tag was included correctly.

Saving and copying

After a QR is generated, three actions are available: Save PNG downloads the QR image, Copy Address copies the bare address string to your clipboard, and Edit returns to the input step so you can correct the address or tag without reopening the modal.

■ QR codes of receive addresses are Category A material — they are public by construction. It is safe to scan them on a networked device after your air-gapped session. Do not use the QR modal to encode private keys or seed phrases.

Chapter 9

Overview

The derivation pipeline is fully deterministic and stateless. Given the same inputs — mnemonic, passphrase, coin, path, script type, and address count — it always produces byte-identical output. The HTML and Python implementations are specification-equivalent and cross-verify each other. No external libraries, no network calls, and no randomness are involved after the mnemonic is established.

The pipeline runs in this order:

1. Validate the mnemonic against the BIP39 word list and verify the checksum.
2. NFKD-normalize the mnemonic and passphrase, then run PBKDF2-HMAC-SHA512 (2,048 iterations) to produce a 64-byte seed.
3. Derive the BIP32 master node via HMAC-SHA512 keyed on 'Bitcoin seed'.
4. Traverse the derivation path using BIP32 CKDpriv for each path component.
5. For each address index on both branches, derive the child key and encode the address for the target script type and coin.

BIP39 validation and the checksum

A 12-word mnemonic encodes 128 bits of entropy plus a 4-bit checksum. A 24-word mnemonic encodes 256 bits plus an 8-bit checksum. The checksum is the first few bits of SHA256 of the entropy bytes. During validation, the tool reconstructs the entropy from the word indices, recomputes the checksum, and compares it to what the final word encodes.

A checksum failure means at least one word is wrong. About 1 in 16 random single-word errors pass the checksum by chance — so a passing checksum is a strong signal but not a guarantee that every word is correct.

Seed derivation: PBKDF2

The mnemonic string (space-separated, NFKD-normalized) is used as the PBKDF2 password. The salt is the string 'mnemonic' followed immediately by the NFKD-normalized passphrase (or just 'mnemonic' if the passphrase is empty). PBKDF2-HMAC-SHA512 runs for 2,048 iterations and produces 64 bytes of seed material.

BIP32 master node

The 64-byte seed is fed into HMAC-SHA512 with the key 'Bitcoin seed'. The first 32 bytes of the output become the master private scalar; the last 32 bytes become the master chain code. The root fingerprint shown in the UI is the first 4 bytes of HASH160 of the compressed master public key.

Child key derivation (CKDpriv)

Each component of the derivation path is applied in sequence using BIP32 CKDpriv. Hardened components (marked with ' or h) fold the parent private key into the HMAC input, making it impossible to derive child keys from the parent public key alone. Non-hardened components use only the parent public key and chain code, which

allows watch-only derivation from an xpub.

Account-level path components (purpose, coin_type, account) are always hardened. Branch and address indices are non-hardened.

Address encoding by script type

P2PKH	RIPEMD160(SHA256(pubkey)), prepend version byte, Base58Check-encode.
P2WPKH-P2SH	Wrap the native SegWit witness program in a P2SH script, HASH160 the script, Base58Check-encode with the P2SH version byte.
P2WPKH	RIPEMD160(SHA256(pubkey)), regroup into 5-bit groups, Bech32-encode with the coin's human-readable part.
P2TR (Taproot)	Negate the private key if y-coordinate is odd (BIP340). Compute TapTweak tagged hash. Add the tweak to get the output key. Bech32m-encode the x-only output public key.
Ethereum	Keccak-256 of the uncompressed 64-byte public key (no prefix). Take the last 20 bytes. Apply EIP-55 mixed-case checksum.
XRP	RIPEMD160(SHA256(compressed pubkey)). Prepend 0x00 version byte. Base58Check-encode using the XRPL-specific alphabet.

Elliptic curve arithmetic

All public key operations use secp256k1, the standard curve for Bitcoin and most derived cryptocurrencies. Point addition and scalar multiplication are implemented natively in both the HTML and Python versions — no external elliptic curve library is used. Public keys are always serialized in compressed form (33 bytes: parity byte followed by the x-coordinate).

Ethereum is the exception at one point: its address derivation uses the uncompressed 64-byte x||y representation without a prefix byte, because Ethereum address derivation requires both coordinates.

Chapter 10

Design goals

The .arc format was designed under two constraints that shaped every decision: zero external dependencies, and byte-level compatibility between the HTML and Python implementations. These constraints ruled out AES-GCM (not available in Python's standard library without third-party packages) and led to the HMAC-SHA512-based construction.

Key derivation

Your password is NFKD-normalized and UTF-8 encoded, then fed into PBKDF2-HMAC-SHA512 with a fresh 32-byte random salt and 1,000,000 iterations. This produces a 64-byte master key, which is never used directly. Instead, two derived keys are computed:

- Encryption key: HMAC-SHA512(master_key, b'Arculus ARC v2 encryption key')
- MAC key: HMAC-SHA512(master_key, b'Arculus ARC v2 authentication key')

The domain-separation labels ensure that the encryption key and the MAC key are cryptographically independent — knowing one tells you nothing about the other.

Stream cipher

Encryption uses HMAC-SHA512 in counter mode. For each 64-byte block *i*, a keystream block is produced as:

```
block_i = HMAC-SHA512(  
    key = enc_key,  
    msg = b'Arculus ARC v2 stream\x00' || nonce || uint64_be(i)  
)
```

The plaintext is XORed with as many keystream bytes as needed. Decryption is identical. The 24-byte random nonce, generated fresh for every export, ensures that two exports of the same mnemonic with the same password produce completely different ciphertexts.

Authentication

The authentication tag covers the ciphertext and every metadata field — format version, KDF algorithm, iteration count, cipher name, salt, nonce, and creation timestamp. This means an attacker cannot alter any field (including the iteration count to weaken the KDF) without causing authentication to fail. The MAC is computed and verified using constant-time comparison to prevent timing side-channel attacks.

■ Authentication is always verified before decryption begins. The tool never decrypts a byte of ciphertext until the MAC has been confirmed. This is the encrypt-then-MAC ordering and it is unconditional.

What the iteration count protects

At 1,000,000 iterations of PBKDF2-HMAC-SHA512, a typical software implementation takes a few seconds per guess. A GPU-equipped attacker can do many thousands of guesses per second. The iteration count slows

every guess linearly. Against a long, randomly generated password (six or more Diceware words), this is comfortable protection. Against a short dictionary-derived password, it is not meaningful — the search space is too small regardless of cost per guess.

Chapter 11

.arc file — external structure

A .arc file is a three-line UTF-8 text file. Line 1 is the armor header. Line 2 is standard base64-encoded JSON. Line 3 is a trailing newline.

```
ARCULUS-ARC-V2
eyJjaXB0ZXIiOnsibmFtZSI6IkhNQUMtU0hBNTEyLUNUUuIiIsIm5vbmNLX2I2NCI6Ii4uLiJ9...
```

After base64 decoding, the inner JSON has these fields:

magic	"ARCULUS-ARC" — identifies the file type
format	"arculus-encrypted-seed-v2"
version	2 (integer)
created_at	UTC ISO 8601 timestamp, e.g. "2026-06-05T14:32:00.000Z"
kdf.name	"PBKDF2"
kdf.hash	"SHA-512"
kdf.iterations	1000000 for new files; minimum 600000 accepted on import
kdf.salt_b64	Base64 of 32 random bytes
cipher.name	"HMAC-SHA512-CTR"
cipher.nonce_b64	Base64 of 24 random bytes
ciphertext_b64	Base64 of encrypted mnemonic JSON
mac_b64	Base64 of 64-byte HMAC-SHA512 authentication tag

Derived-output JSON

JSON export preserves the full hierarchy. The top-level object has: coin, network, mnemonic, word_count, derivation, account_script_type_used, root_fingerprint, and an accounts array. Each account has extended keys at root and account level, plus receiving and change arrays of address records.

Each address record contains path, branch, address, public_key_hex, private_key_hex, and private_key_wif. Taproot records add internal_public_key_hex, tweak_hex, output_public_key_hex, output_private_key_hex, output_private_key_wif, and output_key_parity. Ethereum records add ethereum_public_key_hex and erc20_address. XRP records use xrp_classic_address instead of address.

CSV export

One row per address, with account-level fields repeated on every row. The first row is a header. Null fields for a given coin or script type appear as empty cells. Values with commas or quotes are RFC 4180-escaped.

TXT export

Human-readable labeled sections. Header block at top, then one section per account with receiving and change subsections. Extended keys appear before the address listing in each account section. Fields are left-aligned with consistent column spacing.

PDF export

Produced by jsPDF (loaded from cdnjs, HTML only; not available in Python CLI). Landscape A4, title block on every page showing coin, version, timestamp, and fingerprint. Extended keys section on the first page, then an address table spanning as many pages as needed. Receiving branch rows appear in green, change in grey.

■ A printed PDF of derived keys is a physical key document. It must be stored with the same physical security as a paper mnemonic backup and must not be printed on a networked printer.

Format version history

Format	Introduced	Produced	Imported
ARCULUS-ARC-V2 (armored)	v1.1.0+	Yes	Yes
JSON V2 (raw)	Pre-v1.1.0	No	Yes
V1 AES-GCM	Early HTML only	No	HTML only
Python V1	Early Python	No	Yes

Chapter 12

About these vectors

The vectors in this chapter are the ground truth for any reimplementation of the derivation engine. Two implementations are considered equivalent if and only if they produce byte-identical output for every vector — matching addresses without matching private keys is not sufficient, because an accidental cancellation of errors could produce a correct address from incorrect key material.

The test mnemonics are the standard BIP39 community test vectors (all-zeros entropy, all-max entropy) used widely in the ecosystem for cross-implementation verification. They control no real funds.

Test mnemonic inputs

TV1 — 12 word, no passphrase	abandon (x11) about
TV1-T — 12 word, passphrase TREZOR	abandon (x11) about / passphrase: TREZOR
TV2 — 24 word, no passphrase	abandon (x23) art
TV3 — 12 word all-max, no passphrase	zoo (x11) wrong

TV1 — Seeds and master nodes

```
# TV1 (empty passphrase) — 64-byte seed:
5eb00bbddcf069084889a8ab9155568165f5c453ccb85e70811aaed6f6da5fc1
9a5ac40b389cd370d086206dec8aa6c43daea6690f20ad3d8d48b2d2ce9e38e4

# TV1-T (passphrase TREZOR) — 64-byte seed:
c55257c360c07c72029aebc1b53c05ed0362ada38ead3e3e9efa3708e5349553
1f09a6987599d18264c1e1c92f2cf141630c7a3c4ab7c81b2f001698e7463b04

# TV1 master private key (IL = HMAC-SHA512('Bitcoin seed', seed)[0:32]):
1837c1be8e2995ec11cda2b066151be2cfb48adf9e47b151d46adab3a21cdf67

# TV1 root fingerprint:
73c5da0a
```

TV1 — Bitcoin P2WPKH (bc1q), m/84'/0'/0'

Path	Address	Private key (WIF, first 12 chars)
m/84'/0'/0'/0/0	bc1qcr8te4kr609gcawutmrza0j4xv80jy8z306fyu	KyZpNDKn...
m/84'/0'/0'/0/1	bc1qnjg0jd8228aq7egyzacy8cys3knf9xvrerkf9g	Kxpf5b8p...
m/84'/0'/0'/0/2	bc1qp59yckz4ae5c4efgw2s5wfyvrz0ala7rgvuz8z	L1WWMeKC...

m/84'/0'/0'/1/0	bc1q8c6fshw2dlwun7ekn9qwf37cu2rn755upcp6e1	KxuoxufJ...
-----------------	--	-------------

TV1 — Bitcoin P2TR (Taproot), m/86'/0'/0'

Index 0 — m/86'/0'/0'/0/0:

address: bc1p5cyxnuuwxmuvkwfem96lqzszd02n6xdcjrs20cac6yqjjwudpxqkedrcr
 internal_public_key: cc8a4bc64d897bddc5fbc2f670f7a8ba0b386779106cf1223c6fc5d7cd6fc115
 tweak: 2ca01ed85cf6b6526f73d39a1111cd80333bdc00ce98992859848a90a6f0258
 output_public_key: a60869f0dbcf1dc659c9cecbaf8050135ea9e8cdc487053f1dc6880949dc684c
 output_key_parity: 1 (odd Y — scalar negation was applied)

TV1 — Ethereum, m/44'/60'/0'

Index 0 — m/44'/60'/0'/0/0:

address (EIP-55): 0x9858EfFD232B4033E47d90003D41EC34EcaEda94
 private_key_hex: 1ab42cc412b618bdea3a599e3c9bae199ebf030895b039e9db1e30dafb12b727

Index 1 — m/44'/60'/0'/0/1:

address: 0x6Fac4D18c912343BF86fa7049364Dd4E424Ab9C0

TV1 — XRP, m/44'/144'/0'

Index 0 — m/44'/144'/0'/0/0:

xrp_classic_address: rHsMGQEkVNJmpGws8XUBoTBiAAbwxZN5v3
 private_key_hex: 90802a50aa84efb6cdb225f17c27616ea94048c179142fecf03f4712a07ea7a4

.arc encryption — deterministic key derivation vector

password: correct horse battery staple
 salt: 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
 iterations: 1000000

enc_key (first 32 bytes):

3c8d5f9435d4d935f8dbcb48285ca8d578079eded9ba046a58fb4a6c6b73e6c

mac_key (first 32 bytes):

d5d4d1a6f5f036d040e502ab0c9759ca517463f9dbd971bc0cf3a27b5a728a9

Chapter 13

About this listing

The following is the complete source of Arculus_Recovery.html (3,626 lines, 162,223 bytes). This single file is the entire HTML application. It has no build step, no external runtime dependencies, and no import statements that fetch anything from the network. Open it directly in a browser.

■ The cryptographic implementations (BIP39, PBKDF2, BIP32, secp256k1 arithmetic, HMAC-SHA512-CTR, QR encoder) are all in this file. The Python implementation in Arculus_Recovery.py uses the same algorithms and produces identical output.

```

1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8" />
5    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6    <title>Arculus Recovery</title>
7    <link rel="icon" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><t
8    <style>
9      ■ :root {
10     ■   --bg: #f7f8fa;
11     ■   --page-top: #eef2ff;
12     ■   --card: #ffffff;
13     ■   --field: #ffffff;
14     ■   --button: #ffffff;
15     ■   --primary: #111827;
16     ■   --primary-text: #ffffff;
17     ■   --text: #111827;
18     ■   --muted: #6b7280;
19     ■   --ok: #1b8f2f;
20     ■   --bad: #b71c1c;
21     ■   --border: #d1d5db;
22     ■   --focus: #2563eb;
23     ■   --shadow: rgba(17, 24, 39, 0.06);
24     ■ }
25     ■ body.dark-mode {
26     ■   --bg: #111827;
27     ■   --page-top: #0b1220;
28     ■   --card: #1f2937;
29     ■   --field: #111827;
30     ■   --button: #374151;
31     ■   --primary: #e5e7eb;
32     ■   --primary-text: #111827;
33     ■   --text: #f9fafb;
34     ■   --muted: #9ca3af;
35     ■   --ok: #4ade80;
36     ■   --bad: #f87171;
37     ■   --border: #4b5563;
38     ■   --focus: #60a5fa;
39     ■   --shadow: rgba(0, 0, 0, 0.35);
40     ■ }
41     ■ body.dark-plus {
42     ■   --bg: #1a1a1a;
43     ■   --page-top: #0d0d0d;

```

```

44 ■      --card: #222222;
45 ■      --field: #1a1a1a;
46 ■      --button: #2e2e2e;
47 ■      --primary: #e86926;
48 ■      --primary-text: #ffffff;
49 ■      --text: #ececce;
50 ■      --muted: #888888;
51 ■      --ok: #5dba6e;
52 ■      --bad: #f07070;
53 ■      --border: #3a3a3a;
54 ■      --focus: #e86926;
55 ■      --shadow: rgba(0, 0, 0, 0.5);
56 ■      --accent: #e86926;
57 ■    }
58 ■    body.dark-plus .app {
59 ■      border-color: #3a3a3a;
60 ■    }

61 ■    body.dark-plus button.primary {
62 ■      background: #e86926;
63 ■      border-color: #e86926;
64 ■      color: #ffffff;
65 ■    }
66 ■    body.dark-plus #status.valid { color: #e86926; }
67 ■    body.dark-plus textarea:focus,
68 ■    body.dark-plus input:focus,
69 ■    body.dark-plus select:focus {
70 ■      outline-color: #e86926;
71 ■      border-color: #e86926;
72 ■    }
73 ■    body.dark-plus .toggle input:checked + .toggle-track {
74 ■      background: #e86926;
75 ■      border-color: #e86926;
76 ■    }
77 ■    body.dark-plus .network-status.online .network-dot { background: #e86926; }
78 ■    * { box-sizing: border-box; }
79 ■    body {
80 ■      margin: 0;
81 ■      font-family: "Segoe UI", Tahoma, Helvetica, Arial, sans-serif;
82 ■      background: linear-gradient(180deg, var(--page-top) 0%, var(--bg) 18%);
83 ■      color: var(--text);
84 ■      min-height: 100dvh;
85 ■      padding: 20px;
86 ■    }
87 ■    .app {
88 ■      max-width: 1400px;
89 ■      margin: 0 auto;
90 ■      background: var(--card);
91 ■      border: 1px solid var(--border);
92 ■      border-radius: 12px;
93 ■      padding: 16px;
94 ■      box-shadow: 0 10px 25px var(--shadow);
95 ■    }
96 ■    .title-row {
97 ■      display: flex;
98 ■      align-items: center;
99 ■      gap: 10px;
100 ■    margin: 0 0 14px;
101 ■  }
102 ■  h1 { margin: 0; font-size: 24px; }
103 ■  .network-status {
104 ■    margin-left: auto;
105 ■    display: inline-flex;

```

```

106     align-items: center;
107     gap: 8px;
108     min-height: 39px;
109     padding: 0 12px;
110     border: 1px solid var(--border);
111     border-radius: 8px;
112     background: var(--field);
113     color: var(--text);
114     font-weight: 700;
115 }
116 .network-dot {
117     width: 12px;
118     height: 12px;
119     border-radius: 999px;
120     background: var(--bad);
121 }
122 .network-status.online .network-dot {
123     background: var(--ok);
124 }
125 .app-version {
126     color: var(--muted);
127     font-size: 13px;
128     font-weight: 700;
129 }
130 .grid { display: grid; grid-template-columns: 260px 1fr; gap: 10px 12px; align-items: start; }
131 .label { font-weight: 600; padding-top: 8px; }
132 textarea, input, select {
133     width: 100%;
134     border: 1px solid var(--border);
135     border-radius: 8px;
136     padding: 10px;
137     font-size: 14px;
138     background: var(--field);
139     color: var(--text);
140 }
141 input[type="radio"], input[type="checkbox"] { width: auto; }
142 textarea:focus, input:focus, select:focus { outline: 2px solid var(--focus); border-color: var(--focus); }
143 #mnemonic { min-height: 60px; resize: vertical; font-family: inherit; }
144 .seed-row { display: flex; gap: 14px; align-items: center; padding-top: 6px; }
145 .words-grid {
146     display: grid;
147     grid-template-columns: repeat(4, minmax(150px, 1fr));
148     gap: 8px;
149 }
150 .word-cell { display: grid; grid-template-columns: 28px 1fr; gap: 6px; align-items: center; }
151 .word-num { color: var(--muted); text-align: right; font-size: 12px; }
152 .valid { color: var(--ok) !important; font-weight: 700 !important; }
153 .invalid { color: var(--bad) !important; font-weight: 700 !important; }
154 .muted { color: var(--muted); }
155 .row-inline { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
156 .buttons { display: flex; gap: 10px; margin-top: 6px; flex-wrap: wrap; }
157 button {
158     border: 1px solid var(--border);
159     border-radius: 8px;
160     background: var(--button);
161     color: var(--text);
162     padding: 10px 14px;
163     font-weight: 600;
164     cursor: pointer;
165 }
166 button.primary { background: var(--primary); color: var(--primary-text); border-color: var(--primary); }
167 button.generate-seed {

```

```
168 ■      background: #22c55e;
169 ■      color: #111827;
170 ■      border-color: #22c55e;
171 ■    }
172 ■    .settings-button {
173 ■      width: 39px;
174 ■      height: 39px;
175 ■      padding: 0;
176 ■      display: inline-flex;
177 ■      align-items: center;
178 ■      justify-content: center;
179 ■      font-size: 20px;
180 ■      line-height: 1;

181 ■    }
182 ■    .settings-button::before {
183 ■      content: "";
184 ■      width: 18px;
185 ■      height: 18px;
186 ■      border: 2px solid currentColor;
187 ■      border-radius: 50%;
188 ■      box-shadow: 0 -7px 0 -5px currentColor, 0 7px 0 -5px currentColor, 7px 0 0 -5px currentColor, -7px 0
189 ■    }
190 ■    .seed-row button {
191 ■      display: inline-flex;
192 ■      align-items: center;
193 ■      justify-content: center;
194 ■      min-height: 39px;
195 ■      font-family: inherit;
196 ■      font-size: 14px;
197 ■      line-height: 1.2;
198 ■      white-space: nowrap;
199 ■    }
200 ■    button:hover { filter: brightness(0.98); }
201 ■    dialog {
202 ■      width: min(360px, calc(100vw - 40px));
203 ■      border: 1px solid var(--border);
204 ■      border-radius: 8px;
205 ■      padding: 0;
206 ■      background: var(--card);
207 ■      color: var(--text);
208 ■      box-shadow: 0 18px 45px rgba(0, 0, 0, 0.25);
209 ■    }
210 ■    dialog::backdrop { background: rgba(17, 24, 39, 0.45); }
211 ■    .settings-panel { padding: 16px; }
212 ■    .settings-header {
213 ■      display: flex;
214 ■      align-items: center;
215 ■      justify-content: space-between;
216 ■      gap: 12px;
217 ■      margin-bottom: 16px;
218 ■    }
219 ■    .settings-header h2 { margin: 0; font-size: 18px; }
220 ■    .settings-close {
221 ■      width: 34px;
222 ■      height: 34px;
223 ■      padding: 0;
224 ■      line-height: 1;
225 ■    }
226 ■    .setting-row {
227 ■      display: flex;
228 ■      align-items: center;
229 ■      justify-content: space-between;
```



```

230 ■      gap: 16px;
231 ■      padding: 12px 0;
232 ■      border-top: 1px solid var(--border);
233 ■    }
234 ■    .toggle {
235 ■      position: relative;
236 ■      display: inline-flex;
237 ■      align-items: center;
238 ■      width: 52px;
239 ■      height: 30px;
240 ■      flex: 0 0 auto;

241 ■    }
242 ■    .toggle input {
243 ■      position: absolute;
244 ■      opacity: 0;
245 ■      pointer-events: none;
246 ■    }
247 ■    .toggle-track {
248 ■      width: 100%;
249 ■      height: 100%;
250 ■      border-radius: 999px;
251 ■      background: #d1d5db;
252 ■      border: 1px solid var(--border);
253 ■      transition: background 0.16s ease;
254 ■    }
255 ■    .toggle-track::after {
256 ■      content: "";
257 ■      position: absolute;
258 ■      top: 4px;
259 ■      left: 4px;
260 ■      width: 22px;
261 ■      height: 22px;
262 ■      border-radius: 50%;
263 ■      background: #ffffff;
264 ■      box-shadow: 0 1px 3px rgba(0, 0, 0, 0.25);
265 ■      transition: transform 0.16s ease;
266 ■    }
267 ■    .toggle input:checked + .toggle-track {
268 ■      background: var(--focus);
269 ■      border-color: var(--focus);
270 ■    }
271 ■    .toggle input:checked + .toggle-track::after { transform: translateX(22px); }
272 ■    .toggle input:focus + .toggle-track { outline: 2px solid var(--focus); outline-offset: 2px; }
273 ■    .button-field {
274 ■      width: 260px;
275 ■      border: 1px solid var(--border);
276 ■      border-radius: 8px;
277 ■      padding: 10px 14px;
278 ■      font-size: 14px;
279 ■      font-weight: 600;
280 ■      background: var(--field);
281 ■      color: var(--text);
282 ■    }
283 ■    .export-format {
284 ■      width: 120px;
285 ■    }
286 ■    #status { margin: 6px 0; min-height: 20px; font-weight: 600; }
287 ■    #output { min-height: 120px; resize: vertical; white-space: pre; overflow: auto; font-family: ui-monospace; }
288 ■    .footer { margin-top: 10px; font-size: 12px; color: var(--muted); }
289 ■    .label-with-tip { display: flex; align-items: center; gap: 6px; padding-top: 8px; }
290 ■    .label-with-tip .label { padding-top: 0; }
291 ■    .info-icon {

```

```
292     display: inline-flex;
293     align-items: center;
294     justify-content: center;
295     width: 16px;
296     height: 16px;
297     border-radius: 50%;
298     border: 1.5px solid var(--muted);
299     color: var(--muted);
300     font-size: 10px;
301
302     font-weight: 700;
303     cursor: default;
304     flex-shrink: 0;
305     position: relative;
306     line-height: 1;
307 }
308 .info-icon:hover .tooltip,
309 .info-icon:focus .tooltip { display: block; }
310 .tooltip {
311     display: none;
312     position: absolute;
313     left: 50%;
314     bottom: calc(100% + 8px);
315     transform: translateX(-50%);
316     width: 260px;
317     background: var(--primary);
318     color: var(--primary-text);
319     font-size: 12px;
320     font-weight: 400;
321     line-height: 1.5;
322     padding: 8px 10px;
323     border-radius: 7px;
324     box-shadow: 0 4px 14px rgba(0,0,0,0.2);
325     z-index: 100;
326     pointer-events: none;
327     white-space: normal;
328 }
329 .tooltip::after {
330     content: '';
331     position: absolute;
332     top: 100%;
333     left: 50%;
334     transform: translateX(-50%);
335     border: 5px solid transparent;
336     border-top-color: var(--primary);
337 }
338 /* Laptop breakpoint: tighten spacing without collapsing to single-column */
339 @media (max-width: 1280px) and (min-width: 981px) {
340     body { padding: 8px; }
341     .app { padding: 10px; }
342     .grid { grid-template-columns: 180px 1fr; gap: 8px 10px; }
343     h1 { font-size: 20px; }
344     #mnemonic { min-height: 60px; }
345     textarea, input, select { padding: 6px 10px; font-size: 13px; }
346     button { padding: 6px 10px; font-size: 13px; }
347     .seed-row button { min-height: 33px; font-size: 13px; }
348     .settings-button { width: 33px; height: 33px; }
349     .network-status { min-height: 33px; padding: 0 10px; font-size: 13px; }
350     .button-field { padding: 6px 10px; font-size: 13px; }
351     .words-grid { grid-template-columns: repeat(4, minmax(120px, 1fr)); gap: 6px; }
352     .word-cell { gap: 4px; }
353 }
```

```
354     @media (max-width: 980px) {
355         body { padding: 8px; }
356         .app { padding: 10px; }
357         .grid { grid-template-columns: 1fr; }
358         .label { padding-top: 0; }
359         .words-grid { grid-template-columns: repeat(2, minmax(120px, 1fr)); }
360     }

361
362     /* ■■ Passphrase toggle ■■ */
363     .passphrase-wrap {
364         position: relative;
365         display: flex;
366         align-items: center;
367     }
368     .passphrase-wrap input { padding-right: 40px; }
369     .toggle-visibility {
370         position: absolute;
371         right: 8px;
372         background: none;
373         border: none;
374         padding: 4px;
375         cursor: pointer;
376         color: var(--muted);
377         display: flex;
378         align-items: center;
379         justify-content: center;
380         border-radius: 4px;
381         min-height: unset;
382         line-height: 1;
383     }
384     .toggle-visibility:hover { filter: none; color: var(--text); }
385
386     /* ■■ Clipboard countdown ■■ */
387     .clipboard-countdown {
388         display: inline;
389         margin-left: 8px;
390         font-size: 11px;
391         font-weight: 700;
392         color: var(--bad);
393     }
394
395     /* ■■ Idle timer banner ■■ */
396     #idleBanner {
397         display: none;
398         background: var(--bad);
399         color: #fff;
400         font-weight: 700;
401         font-size: 13px;
402         padding: 7px 14px;
403         border-radius: 8px;
404         margin-bottom: 10px;
405         text-align: center;
406     }
407     #idleBanner.visible { display: block; }
408
409     /* ■■ Progress bar ■■ */
410     #deriveProgress {
411         display: none;
412         height: 6px;
413         border-radius: 999px;
414         background: var(--border);
415         margin: 6px 0 4px;
```

```
416     overflow: hidden;
417 }
418 #deriveProgress.visible { display: block; }
419 #deriveProgressBar {
420     height: 100%;
421
422     width: 0%;
423     background: var(--focus);
424     border-radius: 999px;
425     transition: width 0.15s ease;
426 }
427 body.dark-plus #deriveProgressBar { background: #e86926; }
428
429 /* ■■ Output tabs ■■ */
430 .output-tabs {
431     display: flex;
432     gap: 4px;
433     margin-top: 8px;
434     margin-bottom: 0;
435 }
436 .output-tab {
437     padding: 5px 12px;
438     border: 1px solid var(--border);
439     border-bottom: none;
440     border-radius: 6px 6px 0 0;
441     background: var(--field);
442     color: var(--muted);
443     font-size: 13px;
444     font-weight: 600;
445     cursor: pointer;
446 }
447 .output-tab.active {
448     background: var(--card);
449     color: var(--text);
450     border-bottom: 1px solid var(--card);
451     margin-bottom: -1px;
452     z-index: 1;
453 }
454 #outputWrap {
455     border: 1px solid var(--border);
456     border-radius: 0 6px 6px 6px;
457     overflow: hidden;
458 }
459
460 /* ■■ Expanded output overlay ■■ */
461 .output-expanded #outputTabs,
462 .output-expanded #outputWrap {
463     position: fixed;
464     left: 0;
465     right: 0;
466     z-index: 900;
467 }
468 .output-expanded #outputTabs {
469     top: 0;
470     margin: 0;
471     padding: 10px 12px 0;
472     background: var(--bg);
473     border-bottom: 1px solid var(--border);
474     border-radius: 0;
475 }
476 .output-expanded #outputWrap {
477     top: 49px;
478     bottom: 0;
```

```
478     border-radius: 0;
479     border-left: none;
480     border-right: none;
481     border-bottom: none;
482     overflow: auto;
483 }
484 .output-expanded #derivedTable { max-height: none; }
485 .output-expanded #output { min-height: calc(100vh - 49px); }
486 .output-expanded #accountKeysPane { min-height: calc(100vh - 49px); }
487
488 /* Backdrop that sits behind the expanded panel */
489 #outputExpandBackdrop {
490     display: none;
491     position: fixed;
492     inset: 0;
493     background: var(--bg);
494     z-index: 899;
495 }
496 .output-expanded #outputExpandBackdrop { display: block; }
497
498 /* Expand button – sits at the far right of the tab bar */
499 .output-tab-expand {
500     margin-left: auto;
501     color: var(--muted);
502     border-color: transparent;
503     background: transparent;
504 }
505 .output-tab-expand:hover { color: var(--text); border-color: var(--border); }
506 #derivedTable {
507     display: none;
508     width: 100%;
509     overflow-x: auto;
510     max-height: 420px;
511     overflow-y: auto;
512 }
513 #derivedTable.active { display: block; }
514 #output { border: none; border-radius: 0; min-height: 120px; }
515 #output.output-hidden { display: none; }
516 .derived-table {
517     width: 100%;
518     border-collapse: collapse;
519     font-size: 12px;
520     font-family: ui-monospace, SFMono-Regular, Menlo, Consolas, monospace;
521 }
522 .derived-table th {
523     position: sticky;
524     top: 0;
525     background: var(--field);
526     color: var(--muted);
527     font-weight: 700;
528     font-size: 11px;
529     text-align: left;
530     padding: 6px 10px;
531     border-bottom: 1px solid var(--border);
532     white-space: nowrap;
533     z-index: 2;
534 }
535 .derived-table td {
536     padding: 5px 10px;
537     border-bottom: 1px solid var(--border);
538     vertical-align: middle;
539     white-space: nowrap;
```

```
540     color: var(--text);
541 }
542 .derived-table td.td-truncate {
543     max-width: 260px;
544     overflow: hidden;
545     text-overflow: ellipsis;
546 }
547 .derived-table td.td-notrunc {
548     max-width: none;
549     overflow: visible;
550 }
551 .derived-table tr:last-child td { border-bottom: none; }
552 .derived-table tr:hover td { background: var(--field); }
553 .branch-badge {
554     font-size: 10px;
555     font-weight: 700;
556     padding: 1px 5px;
557     border-radius: 3px;
558 }
559 .branch-receiving { background: rgba(27,143,47,0.12); color: var(--ok); }
560 .branch-change     { background: rgba(107,114,128,0.12); color: var(--muted); }
561 .copy-row-btn {
562     border: 1px solid var(--border);
563     border-radius: 4px;
564     background: var(--button);
565     color: var(--muted);
566     padding: 2px 7px;
567     font-size: 10px;
568     font-weight: 700;
569     cursor: pointer;
570     white-space: nowrap;
571     min-height: unset;
572     line-height: 1.4;
573     display: inline-block;
574 }
575 .copy-row-btn:hover { filter: none; color: var(--text); background: var(--field); }
576 .copy-row-btn.copied { color: var(--ok); border-color: var(--ok); }
577 .td-copy-wrap {
578     display: flex;
579     align-items: center;
580     gap: 6px;
581 }
582 .copy-cell-btn {
583     border: 1px solid var(--border);
584     border-radius: 4px;
585     background: var(--button);
586     color: var(--muted);
587     padding: 1px 6px;
588     font-size: 10px;
589     font-weight: 700;
590     cursor: pointer;
591     white-space: nowrap;
592     min-height: unset;
593     line-height: 1.4;
594     flex-shrink: 0;
595 }
596 .copy-cell-btn:hover { filter: none; color: var(--text); background: var(--field); }
597 .copy-cell-btn.copied { color: var(--ok); border-color: var(--ok); }
598
599 #accountKeysPane {
600     display: none;
```

```
601     padding: 16px 20px;
602     font-family: ui-monospace, SFMono-Regular, Menlo, Consolas, monospace;
603     font-size: 12px;
604 }
605 #accountKeysPane.active { display: block; }
606 .ak-title {
607     font-family: "Segoe UI", Tahoma, Helvetica, Arial, sans-serif;
608     font-size: 11px;
609     font-weight: 700;
610     text-transform: uppercase;
611     letter-spacing: 0.06em;
612     color: var(--muted);
613     margin-bottom: 10px;
614 }
615 .ak-grid {
616     display: grid;
617     grid-template-columns: max-content 1fr;
618     gap: 4px 14px;
619     align-items: baseline;
620 }
621 .ak-label {
622     color: var(--muted);
623     font-size: 11px;
624     font-weight: 700;
625     white-space: nowrap;
626     user-select: none;
627 }
628 .ak-value {
629     color: var(--text);
630     font-size: 11.5px;
631     word-break: break-all;
632     display: flex;
633     align-items: baseline;
634     gap: 6px;
635 }
636 .ak-value span { flex: 1; }
637 .ak-copy {
638     border: 1px solid var(--border);
639     border-radius: 4px;
640     background: var(--button);
641     color: var(--muted);
642     padding: 1px 6px;
643     font-size: 10px;
644     font-weight: 700;
645     cursor: pointer;
646     white-space: nowrap;
647     min-height: unset;
648     line-height: 1.5;
649     flex-shrink: 0;
650     font-family: "Segoe UI", Tahoma, Helvetica, Arial, sans-serif;
651 }
652 .ak-copy:hover { filter: none; color: var(--text); background: var(--field); }
653 .ak-copy.copied { color: var(--ok); border-color: var(--ok); }
654 .ak-divider {
655     grid-column: 1 / -1;
656     height: 1px;
657     background: var(--border);
658     margin: 4px 0;
659 }
660 #qrModal {
661     width: min(340px, calc(100vw - 40px));
662     border: 1px solid var(--border);
```

```

663     border-radius: 12px;
664     padding: 0;
665     background: var(--card);
666     color: var(--text);
667     box-shadow: 0 18px 45px rgba(0,0,0,0.28);
668 }
669 #qrModal::backdrop { background: rgba(17,24,39,0.5); }
670 .qr-panel { padding: 18px; display: flex; flex-direction: column; align-items: center; gap: 14px; }
671 .qr-header { width: 100%; display: flex; align-items: center; justify-content: space-between; gap: 12px; }
672 .qr-header h2 { margin: 0; font-size: 17px; }
673 .qr-address-label {
674     font-size: 11px;
675     color: var(--muted);
676     word-break: break-all;
677     text-align: center;
678     max-width: 100%;
679     line-height: 1.5;
680 }
681 #qrCanvas {
682     border-radius: 8px;
683     image-rendering: pixelated;
684 }
685 .qr-actions { display: flex; gap: 10px; }
686 button.qr-trigger {
687     padding: 2px 8px;
688     font-size: 12px;
689     font-weight: 600;
690     min-height: 26px;
691     border-radius: 6px;
692 }
693 </style>
694 </head>
695 <body>
696     <div class="app">
697         <div class="title-row">
698             <h1>Arculus Recovery</h1>
699             <span class="app-version">v1.5.0</span>
700             <button id="settingsBtn" type="button" class="settings-button" aria-label="Open settings" title="Settings">⚙️</button>
701             <div id="networkStatus" class="network-status" title="Browser network status" aria-live="polite">
702                 <span class="network-dot" aria-hidden="true"></span>
703                 <span>Network</span>
704             </div>
705         </div>
706         <div class="grid">
707             <div class="label">Mnemonic (12 or 24 words)</div>
708             <textarea id="mnemonic" spellcheck="false" placeholder="Paste mnemonic here..."></textarea>
709
710             <div></div>
711             <div class="seed-row">
712                 <span class="muted">Seed length:</span>
713                 <label><input type="radio" name="seedLen" value="12" checked> 12 words</label>
714                 <label><input type="radio" name="seedLen" value="24"> 24 words</label>
715                 <button id="copySeedBtn" type="button" class="copy-seed">Copy Seed</button>
716                 <button id="generateSeedBtn" type="button" class="generate-seed">Generate Random Seed</button>
717                 <button id="showSeedBtn" type="button">Show Seed</button>
718                 <button id="clearAllBtn" type="button">Clear All</button>
719             </div>
720
721             <div class="label">Numbered Words</div>
722             <div class="words-grid" id="wordsGrid"></div>
723
724             <div class="label">Passphrase</div>

```



```

725     <div class="passphrase-wrap">
726         <input id="passphrase" type="password" autocomplete="off" spellcheck="false" />
727         <button type="button" class="toggle-visibility" id="passphraseToggle" aria-label="Show passphrase" t
728             <svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24" fill="none" str
729         </button>
730     </div>
731
732     <div class="label">Coin</div>
733     <select id="coin">
734         <option selected>Bitcoin</option>
735         <option>Litecoin</option>
736         <option>Dogecoin</option>
737         <option>Ethereum / ERC-20</option>
738         <option>XRP</option>
739     </select>
740
741     <div class="label-with-tip">
742         <span class="label">Derivation Path</span>
743         <span class="info-icon" id="derivationInfoIcon" tabindex="0" aria-label="Derivation path help" style
744     </div>
745     <input id="derivation" type="text" value="m/0'" spellcheck="false" />
746
747     <div class="label">Script Type</div>
748     <select id="scriptType">
749         <option>Auto</option>
750         <option>P2PKH</option>
751         <option>P2WPKH-P2SH</option>
752         <option selected>P2WPKH</option>
753         <option>P2TR</option>
754     </select>
755
756     <div class="label">Address Count</div>
757     <div style="display:flex;align-items:center;gap:12px;">
758         <input id="count" type="number" min="1" step="1" value="5" maxLength="15" style="width:100px;" />
759         <div class="label" style="padding-top:0;white-space:nowrap;">Range</div>
760         <input id="rangeInput" type="text" spellcheck="false" autocomplete="off"
761             maxLength="20" placeholder="e.g. 100-200" style="width:130px;" />
762     </div>
763 </div>
764
765 <div class="buttons">
766     <button id="validateBtn">Validate Mnemonic</button>
767     <button id="deriveBtn">Derive Keys + Addresses</button>
768     <button id="exportBtn">Export</button>
769     <select id="exportFormat" class="export-format" aria-label="Export format">
770         <option>JSON</option>
771         <option>CSV</option>
772         <option>TXT</option>
773         <option selected>PDF</option>
774     </select>
775     <button id="encryptExportBtn">Encrypt/Export Seed</button>
776     <button id="importSeedBtn">Import Seed</button>
777
778     <button id="qrAddressesBtn">QR Export</button>
779     <input id="rootFingerprintDisplay" class="button-field" type="text" value="Root Fingerprint:" readonly
780 </div>
781
782 <div id="idleBanner" role="alert"></div>
783 <div id="status" aria-live="polite">Ready</div>
784 <div id="deriveProgress" aria-hidden="true"><div id="deriveProgressBar"></div></div>
785 <div id="outputExpandBackdrop"></div>
786 <div class="output-tabs" id="outputTabs" style="display:none;">

```

```

787     <button type="button" class="output-tab active" data-tab="table">Table View</button>
788     <button type="button" class="output-tab" data-tab="raw">Raw JSON</button>
789     <button type="button" class="output-tab" data-tab="keys">Extended Keys</button>
790     <button type="button" id="expandOutputBtn" class="output-tab output-tab-expand" title="Expand output p
791 </div>
792 <div id="outputWrap">
793     <div id="derivedTable"></div>
794     <textarea id="output" spellcheck="false" readonly></textarea>
795     <div id="accountKeysPane" aria-label="Account extended keys"></div>
796 </div>
797 <div class="footer">Runs fully offline in your browser. Never share real seed phrases.</div>
798 <input id="importSeedFile" type="file" accept=".arc,.json,application/json" hidden />
799 <dialog id="qrModal" aria-labelledby="qrModalTitle">
800     <div class="qr-panel">
801         <div class="qr-header">
802             <h2 id="qrModalTitle">QR Export</h2>
803             <button id="qrModalCloseBtn" type="button" class="settings-close" aria-label="Close QR">x</button>
804         </div>
805         <!-- Step 1: address input (always shown) -->
806         <div style="width:100%;display:flex;gap:8px;align-items:stretch;">
807             <input id="qrAddressInput" type="text" spellcheck="false" autocomplete="off"
808                 placeholder="Paste address here..." style="flex:1;min-width:0;" />
809             <button id="qrGenerateBtn" type="button" class="primary">Generate</button>
810         </div>
811
812         <!-- Step 2: XRP memo prompt (shown only for XRP addresses, before QR renders) -->
813         <div id="qrMemoPrompt" style="display:none;width:100%;border:1px solid var(--border);border-radius:8
814             <div style="font-size:13px;font-weight:700;margin-bottom:4px;">Destination Tag / Memo</div>
815             <div style="font-size:12px;color:var(--muted);margin-bottom:12px;">
816                 Some exchanges (e.g. Robinhood) require a destination tag to credit your account correctly.
817                 Enter yours below, or skip to generate a plain address QR code.
818             </div>
819             <input id="qrMemoInput" type="text" inputmode="numeric" spellcheck="false" autocomplete="off"
820                 placeholder="Destination tag (numbers only)" style="width:100%;box-sizing:border-box;margin-bott
821             <div style="display:flex;gap:8px;">
822                 <button id="qrMemoConfirmBtn" type="button" class="primary" style="flex:1;">Generate QR</button>
823                 <button id="qrMemoSkipBtn" type="button" style="flex:1;">Skip – No Tag</button>
824             </div>
825         </div>
826
827         <!-- QR output -->
828         <canvas id="qrCanvas" width="256" height="256" style="display:none;"></canvas>
829         <div id="qrAddressLabel" class="qr-address-label" style="display:none;"></div>
830         <div class="qr-actions" id="qrSaveRow" style="display:none;">
831             <button id="qrSaveBtn" type="button">Save PNG</button>
832             <button id="qrCopyAddrBtn" type="button">Copy Address</button>
833             <button id="qrEditBtn" type="button">Edit</button>
834         </div>
835     </div>
836 </dialog>
837
838 <dialog id="settingsDialog" aria-labelledby="settingsTitle">
839     <div class="settings-panel">
840         <div class="settings-header">
841             <h2 id="settingsTitle">Settings</h2>
842             <button id="settingsCloseBtn" type="button" class="settings-close" aria-label="Close settings">x</button>
843         </div>
844         <div class="setting-row" style="flex-direction:column; align-items:flex-start; gap:10px;">
845             <span style="font-weight:600;">Theme</span>
846             <div style="display:flex; gap:8px; flex-wrap:wrap;">
847                 <label style="display:flex;align-items:center;gap:6px;cursor:pointer;">
848                     <input type="radio" name="themeMode" id="themeModeLight" value="light"> Light

```

```

849         </label>
850         <label style="display:flex;align-items:center;gap:6px;cursor:pointer;">
851             <input type="radio" name="themeMode" id="themeModeDark" value="dark"> Dark
852         </label>
853         <label style="display:flex;align-items:center;gap:6px;cursor:pointer;">
854             <input type="radio" name="themeMode" id="themeModeDarkPlus" value="dark-plus"> Dark+
855         </label>
856     </div>
857 </div>
858
859 <!-- Advanced section -->
860 <div class="setting-row" style="flex-direction:column;align-items:flex-start;gap:0;padding-bottom:0;">
861     <button type="button" id="advancedToggleBtn" aria-expanded="false"
862         style="background:none;border:none;padding:4px 0;font-weight:600;font-size:14px;cursor:pointer;">
863         <span id="advancedChevron" style="display:inline-block;transition:transform 0.18s;">■</span>
864         Advanced
865     </button>
866 </div>
867 <div id="advancedSection" style="display:none;">
868     <div class="setting-row">
869         <div style="display:flex;flex-direction:column;gap:3px;">
870             <span style="font-weight:600;">Auto-Derive on Settings Change</span>
871             <span style="font-size:12px;color:var(--muted);">Automatically re-derives when coin, path, cou
872         </div>
873         <label class="toggle" aria-label="Auto-derive on settings change">
874             <input type="checkbox" id="autoderiveToggle" />
875             <span class="toggle-track"></span>
876         </label>
877     </div>
878 </div>
879
880 </div>
881 </dialog>
882 </div>
883
884 <script>
885     const APP_VERSION = '1.5.0';
886     const BIP39_WORDS = ['abandon','ability','able','about','above','absent','absorb','abstract','absurd','a
887     const BIP39_SET = new Set(BIP39_WORDS);
888     const BIP39_INDEX = Object.fromEntries(BIP39_WORDS.map((w, i) => [w, i]));
889
890     const HARDENED = 0x80000000;
891     const P = BigInt('0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFC2F');
892     const N = BigInt('0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141');
893     const Gx = BigInt('55066263022277343669578718895168534326250603453777594175500187360389116729240');
894     const Gy = BigInt('32670510020758816978083085130507043184471273380659243275938904335757337482424');
895     const G = { x: Gx, y: Gy };
896
897     const NETS = {
898         bitcoin: {
899             p2pkh: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
900             'p2wpkh-p2sh': { xprv: 0x049D7878, xpub: 0x049D7CB2 },
901
902             p2wpkh: { xprv: 0x04B2430C, xpub: 0x04B24746 },
903             p2tr: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
904             wif: 0x80,
905             p2pkhPrefix: 0x00,
906             p2shPrefix: 0x05,
907             hrp: 'bc',
908             networkName: 'mainnet'
909         },
910         litecoin: {
911             p2pkh: { xprv: 0x019D9CFE, xpub: 0x019DA462 },

```

```

911     'p2wpkh-p2sh': { xprv: 0x01B26792, xpub: 0x01B26EF6 },
912     p2wpkh: { xprv: 0x04B2430C, xpub: 0x04B24746 },
913     p2tr: { xprv: 0x019D9CFE, xpub: 0x019DA462 },
914     wif: 0xB0,
915     p2pkhPrefix: 0x30,
916     p2shPrefix: 0x32,
917     hrp: 'ltc',
918     networkName: 'mainnet'
919 },
920 dogecoin: {
921     // Dogecoin BIP32 versions (dgpv/dgub). No common SLIP-0132 y/z versions.
922     p2pkh: { xprv: 0x02FAC398, xpub: 0x02FACAFD },
923     'p2wpkh-p2sh': { xprv: 0x02FAC398, xpub: 0x02FACAFD },
924     p2wpkh: { xprv: 0x02FAC398, xpub: 0x02FACAFD },
925     p2tr: null,
926     wif: 0x9E,
927     p2pkhPrefix: 0x1E,
928     p2shPrefix: 0x16,
929     hrp: 'doge',
930     networkName: 'mainnet'
931 },
932 ethereum: {
933     p2pkh: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
934     'p2wpkh-p2sh': { xprv: 0x0488ADE4, xpub: 0x0488B21E },
935     p2wpkh: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
936     p2tr: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
937     addressFamily: 'ethereum',
938     networkName: 'mainnet'
939 },
940 xrp: {
941     p2pkh: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
942     'p2wpkh-p2sh': { xprv: 0x0488ADE4, xpub: 0x0488B21E },
943     p2wpkh: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
944     p2tr: { xprv: 0x0488ADE4, xpub: 0x0488B21E },
945     addressFamily: 'xrp',
946     networkName: 'mainnet'
947 }
948 };
949 const DEFAULT_ACCOUNT_DERIVATION = {
950     bitcoin: "m/0'",
951     litecoin: "m/84'/2'/0'",
952     dogecoin: "m/44'/3'/0'",
953     ethereum: "m/44'/60'/0'",
954     xrp: "m/44'/144'/0'"
955 };
956
957 const B58 = '123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz';
958 const XRP_B58 = 'rpshnaf39wBUDNEGHJKLM4PQRST7VWXYZ2bcdeCg65jkm8oFqi1tuvAxyz';
959 const BECH32 = 'qpzry9x8gf2tvdw0s3jn54khce6mua7l';
960
961 const els = {
962     mnemonic: document.getElementById('mnemonic'),
963     wordsGrid: document.getElementById('wordsGrid'),
964     passphrase: document.getElementById('passphrase'),
965     copySeedBtn: document.getElementById('copySeedBtn'),
966     generateSeedBtn: document.getElementById('generateSeedBtn'),
967     coin: document.getElementById('coin'),
968     derivation: document.getElementById('derivation'),
969     scriptType: document.getElementById('scriptType'),
970     count: document.getElementById('count'),
971     rangeInput: document.getElementById('rangeInput'),
972     status: document.getElementById('status'),

```

```

973 output: document.getElementById('output'),
974 settingsBtn: document.getElementById('settingsBtn'),
975 networkStatus: document.getElementById('networkStatus'),
976 settingsDialog: document.getElementById('settingsDialog'),
977 settingsCloseBtn: document.getElementById('settingsCloseBtn'),
978 validateBtn: document.getElementById('validateBtn'),
979 deriveBtn: document.getElementById('deriveBtn'),
980 exportBtn: document.getElementById('exportBtn'),
981 exportFormat: document.getElementById('exportFormat'),
982 encryptExportBtn: document.getElementById('encryptExportBtn'),
983 importSeedBtn: document.getElementById('importSeedBtn'),
984 showSeedBtn: document.getElementById('showSeedBtn'),
985 rootFingerprintDisplay: document.getElementById('rootFingerprintDisplay'),
986 importSeedFile: document.getElementById('importSeedFile'),
987 themeModeRadios: () => document.querySelectorAll('input[name="themeMode"]'),
988 themeModeLight: document.getElementById('themeModeLight'),
989 themeModeDark: document.getElementById('themeModeDark'),
990 themeModeDarkPlus: document.getElementById('themeModeDarkPlus'),
991 clearAllBtn: document.getElementById('clearAllBtn')
992 };
993
994 const wordInputs = [];
995 let syncBusy = false;
996 let importedSeedState = null;
997 let showingImportedSeed = false;
998 let rootFingerprintRefreshToken = 0;
999 let lastDerivedOutput = null;
1000 const ARC_MAGIC = 'ARCULUS-ARC';
1001 const ARC_ARMOR_HEADER = 'ARCULUS-ARC-V2';
1002 const ARC_V2_FORMAT = 'arculus-encrypted-seed-v2';
1003 const ARC_V2_KDF_ITERATIONS = 1000000;
1004 const ARC_V2_MIN_KDF_ITERATIONS = 600000;
1005 const ARC_V2_SALT_BYTES = 32;
1006 const ARC_V2_NONCE_BYTES = 24;
1007 const IDLE_TIMEOUT_MS = 5 * 60 * 1000; // 5 minutes
1008 const CLIPBOARD_CLEAR_SECS = 60;
1009
1010 // ■■ Idle timer ████████████████████████████████████████████████████████████████
1011 let idleTimer = null;
1012 let idleWarningTimer = null;
1013
1014 function resetIdleTimer() {
1015   clearTimeout(idleTimer);
1016   clearTimeout(idleWarningTimer);
1017   const banner = document.getElementById('idleBanner');
1018   if (banner) banner.classList.remove('visible');
1019   idleWarningTimer = setTimeout(() => {
1020     const b = document.getElementById('idleBanner');
1021
1022     if (b) {
1023       b.textContent = 'Idle timeout in 60 seconds. All fields will be cleared.';
1024       b.classList.add('visible');
1025     }
1026   }, IDLE_TIMEOUT_MS - 60000);
1027   idleTimer = setTimeout(() => {
1028     handleClearAll();
1029     const b = document.getElementById('idleBanner');
1030     if (b) {
1031       b.textContent = 'Session cleared due to inactivity.';
1032       b.classList.add('visible');
1033       setTimeout(() => b.classList.remove('visible'), 5000);
1034     }
1035   }, IDLE_TIMEOUT_MS);

```

[illegible]

```

1097     return rows.some(function(r) { return r[k] !== null && r[k] !== undefined && r[k] !== ''; });
1098 });
1099
1100 // Columns that should appear first (promoted), with no truncation
1101 const PROMOTED = ['branch', 'path', 'address', 'private_key_wif'];
1102 const NO_TRUNCATE = new Set(['address', 'private_key_wif', 'public_key_hex', 'private_key_hex', 'ether
1103
1104 // Build final ordered column list: promoted cols first (if present), then the rest
1105 const promotedPresent = PROMOTED.filter(function(k) { return allCols.includes(k); });
1106 const rest = allCols.filter(function(k) { return !new Set(PROMOTED).has(k); });
1107 const cols = promotedPresent.concat(rest);
1108
1109 const table = document.createElement('table');
1110 table.className = 'derived-table';
1111 const thead = table.createTHead();
1112 const hrow = thead.insertRow();
1113 cols.forEach(function(col) {
1114     const th = document.createElement('th');
1115     th.textContent = col.replace(/_/g, ' ');
1116     th.title = col;
1117     hrow.appendChild(th);
1118 });
1119
1120 function makeCopyBtn(val, cls) {
1121     const btn = document.createElement('button');
1122     btn.className = cls || 'copy-cell-btn';
1123     btn.textContent = '■';
1124     btn.title = 'Copy to clipboard';
1125     btn.addEventListener('click', async function() {
1126         try {
1127             await copyTextToClipboard(String(val));
1128             btn.textContent = '✓';
1129             btn.classList.add('copied');
1130             setTimeout(function() { btn.textContent = '■'; btn.classList.remove('copied'); }, 1500);
1131         } catch(e) {}
1132     });
1133     return btn;
1134 }
1135
1136 function makeQrBtn(address) {
1137     const btn = document.createElement('button');
1138     btn.className = 'copy-cell-btn';
1139     btn.textContent = '■';
1140     btn.title = 'Show QR code';
1141
1142     btn.addEventListener('click', function() {
1143         const input = document.getElementById('qrAddressInput');
1144         const memoInput = document.getElementById('qrMemoInput');
1145         const dlg = document.getElementById('qrModal');
1146         resetQrOutput();
1147         input.value = address;
1148         memoInput.value = '';
1149         if (typeof dlg.showModal === 'function') dlg.showModal(); else dlg.setAttribute('open', '');
1150         if (isXrpAddress(address)) {
1151             // XRP: show memo prompt step before rendering
1152             document.getElementById('qrMemoPrompt').style.display = '';
1153             setTimeout(() => memoInput.focus(), 50);
1154         } else {
1155             renderQrFromAddress(address, '');
1156         }
1157     });
1158     return btn;
1159 }

```

```

1159
1160     const tbody = table.createTBody();
1161     rows.forEach(function(row) {
1162         const tr = tbody.insertRow();
1163         cols.forEach(function(col) {
1164             const td = tr.insertCell();
1165             const val = row[col];
1166             if (val === null || val === undefined) { td.className = 'td-truncate'; td.textContent = ''; return; }
1167             if (col === 'branch') {
1168                 td.className = 'td-truncate';
1169                 const badge = document.createElement('span');
1170                 badge.className = 'branch-badger ' + (val === 'receiving' ? 'branch-receiving' : 'branch-change');
1171                 badge.textContent = val;
1172                 td.appendChild(badge);
1173             } else if (NO_TRUNCATE.has(col)) {
1174                 td.className = 'td-notrunc';
1175                 const wrap = document.createElement('span');
1176                 wrap.className = 'td-copy-wrap';
1177                 const text = document.createElement('span');
1178                 text.textContent = String(val);
1179                 wrap.appendChild(makeCopyBtn(val));
1180                 if (col === 'address') wrap.appendChild(makeQrBtn(val));
1181                 wrap.appendChild(text);
1182                 td.appendChild(wrap);
1183                 td.title = String(val);
1184             } else {
1185                 td.className = 'td-truncate';
1186                 td.textContent = String(val);
1187                 td.title = String(val);
1188             }
1189         });
1190     });
1191     return table;
1192 }
1193
1194 function showDerivedTable(data) {
1195     const container = document.getElementById('derivedTable');
1196     const tabs = document.getElementById('outputTabs');
1197     if (!container || !tabs) return;
1198     container.innerHTML = '';
1199     const table = buildDerivedTable(data);
1200     if (table) {
1201         container.appendChild(table);
1202         tabs.style.display = '';
1203         setOutputTab('table');
1204     }
1205     showAccountKeys(data);
1206 }
1207
1208 function setOutputTab(tab) {
1209     currentOutputMode = tab;
1210     const tableEl = document.getElementById('derivedTable');
1211     const rawEl = document.getElementById('output');
1212     const keysEl = document.getElementById('accountKeysPane');
1213     document.querySelectorAll('.output-tab').forEach(function(btn) {
1214         btn.classList.toggle('active', btn.dataset.tab === tab);
1215     });
1216     tableEl.classList.toggle('active', tab === 'table');
1217     rawEl.classList.toggle('output-hidden', tab !== 'raw');
1218     if (keysEl) keysEl.classList.toggle('active', tab === 'keys');
1219 }
1220

```



```
1221     function resetOutputToRaw() {
1222         const tabs = document.getElementById('outputTabs');
1223         const tableEl = document.getElementById('derivedTable');
1224         const rawEl = document.getElementById('output');
1225         if (tabs) tabs.style.display = 'none';
1226         if (tableEl) { tableEl.innerHTML = ''; tableEl.classList.remove('active'); }
1227         if (rawEl) rawEl.classList.remove('output-hidden');
1228         hideAccountKeys();
1229     }
1230
1231     // Key labels: human-readable name for each extended key field
1232     const AK_LABELS = {
1233         root_xprv: 'root xprv',  root_xpub: 'root xpub',
1234         root_yprv: 'root yprv',  root_ypub: 'root ypub',
1235         root_zprv: 'root zprv',  root_zpub: 'root zpub',
1236         root_trprv: 'root trprv', root_trpub: 'root trpub',
1237         account_xprv: 'account xprv', account_xpub: 'account xpub',
1238         account_yprv: 'account yprv', account_ypub: 'account ypub',
1239         account_zprv: 'account zprv', account_zpub: 'account zpub',
1240         account_trprv: 'account trprv', account_trpub: 'account trpub',
1241     };
1242     const AK_KEY_ORDER = Object.keys(AK_LABELS);
1243
1244     function showAccountKeys(data) {
1245         const pane = document.getElementById('accountKeysPane');
1246         if (!pane) return;
1247         pane.innerHTML = '';
1248
1249         const account = (data.accounts || [])[0];
1250         if (!account) return;
1251
1252         const pairs = AK_KEY_ORDER
1253             .filter(function(k) { return account[k]; })
1254             .map(function(k) { return { key: k, label: AK_LABELS[k], value: account[k] }; });
1255
1256         if (!pairs.length) return;
1257
1258         const title = document.createElement('div');
1259         title.className = 'ak-title';
1260         title.textContent = 'Extended Keys';
1261
1262         pane.appendChild(title);
1263
1264         const grid = document.createElement('div');
1265         grid.className = 'ak-grid';
1266         pane.appendChild(grid);
1267
1268         let lastGroup = null;
1269         pairs.forEach(function(pair) {
1270             const group = pair.key.startsWith('root_') ? 'root' : 'account';
1271             if (lastGroup !== null && group !== lastGroup) {
1272                 const div = document.createElement('div');
1273                 div.className = 'ak-divider';
1274                 grid.appendChild(div);
1275             }
1276             lastGroup = group;
1277
1278             const labelEl = document.createElement('div');
1279             labelEl.className = 'ak-label';
1280             labelEl.textContent = pair.label;
1281
1282             const valWrap = document.createElement('div');
1283             valWrap.className = 'ak-value';
```

```

1283
1284     const copyBtn = document.createElement('button');
1285     copyBtn.className = 'ak-copy';
1286     copyBtn.textContent = '■';
1287     copyBtn.title = 'Copy ' + pair.label;
1288     (function(v, btn) {
1289         btn.addEventListener('click', async function() {
1290             try {
1291                 await copyTextToClipboard(v);
1292                 btn.textContent = '✓';
1293                 btn.classList.add('copied');
1294                 setTimeout(function() { btn.textContent = '■'; btn.classList.remove('copied'); }, 1500);
1295             } catch(e) {}
1296         });
1297     })(pair.value, copyBtn);
1298
1299     const valSpan = document.createElement('span');
1300     valSpan.textContent = pair.value;
1301     valSpan.title = pair.value;
1302
1303     valWrap.appendChild(copyBtn);
1304     valWrap.appendChild(valSpan);
1305     grid.appendChild(labelEl);
1306     grid.appendChild(valWrap);
1307 });
1308 }
1309
1310 function hideAccountKeys() {
1311     const pane = document.getElementById('accountKeysPane');
1312     if (pane) { pane.classList.remove('active'); pane.innerHTML = ''; }
1313 }
1314
1315
1316
1317 function getSeedLen() {
1318     return Number(document.querySelector('input[name="seedLen"]:checked').value);
1319 }
1320
1321
1322 function setSeedLen(v) {
1323     const r = document.querySelector(`input[name="seedLen"][value="${v}"]`);
1324     if (r) r.checked = true;
1325     refreshWordEnabled();
1326 }
1327
1328 function normalizeNFKD(s) {
1329     return (s ?? '').normalize('NFKD');
1330 }
1331
1332 // Match Python tool behavior exactly:
1333 // normalize_nfkd(" ".join(mnemonic.strip().split()).split(" "))
1334 function normalizeMnemonicWords(text) {
1335     const normalized = normalizeNFKD((text ?? '').trim().split(/\s+/).filter(Boolean).join(' '));
1336     return normalized === ' ' ? [] : normalized.split(' ');
1337 }
1338
1339 function splitWords(text) {
1340     return normalizeMnemonicWords(text);
1341 }
1342
1343 function uiStatus(msg, cls = '') {
1344     els.status.textContent = msg;
1345     els.status.className = cls;

```

```

1345     }
1346
1347     function applyTheme(mode) {
1348         // mode: 'light' | 'dark' | 'dark-plus'
1349         document.body.classList.remove('dark-mode', 'dark-plus');
1350         if (mode === 'dark') document.body.classList.add('dark-mode');
1351         else if (mode === 'dark-plus') document.body.classList.add('dark-plus');
1352         // sync radio buttons
1353         document.querySelectorAll('input[name="themeMode"]').forEach(r => {
1354             r.checked = r.value === mode;
1355         });
1356         try {
1357             localStorage.setItem('arculusTheme', mode);
1358         } catch {
1359             // Local files may be opened in privacy-restricted contexts.
1360         }
1361     }
1362     // Keep legacy name as alias for any internal callers
1363     function applyDarkMode(enabled) {
1364         applyTheme(enabled ? 'dark' : 'light');
1365     }
1366
1367     function refreshNetworkStatus() {
1368         const online = navigator.onLine;
1369         els.networkStatus.classList.toggle('online', online);
1370         els.networkStatus.title = online ? 'Network route detected' : 'No network connection detected';
1371         els.networkStatus.setAttribute('aria-label', online ? 'Network connected' : 'Network disconnected');
1372     }
1373
1374     function setRootFingerprintDisplay(fingerprint = '') {
1375         els.rootFingerprintDisplay.value = fingerprint ? `Root Fingerprint: ${fingerprint}` : 'Root Fingerprint';
1376     }
1377
1378     function getActiveMnemonic() {
1379         return importedSeedState?.mnemonic || els.mnemonic.value.trim();
1380     }
1381
1382     function protectedSeedStatus() {
1383         return importedSeedState?.kind === 'generated'
1384             ? 'Generated seed phrase is hidden but ready for validation, copy, export, and derivation.'
1385             : 'Encrypted seed imported. Seed phrase is hidden but ready for validation and derivation.';
1386     }
1387
1388     function protectedSeedPlaceholder() {
1389         return importedSeedState?.kind === 'generated'
1390             ? 'Generated seed phrase hidden. Hold Show Seed to reveal.'
1391             : 'Encrypted seed loaded. Hold Show Seed to reveal.';
1392     }
1393
1394     function hasVisibleSeedInput() {
1395         if (els.mnemonic.value.trim()) return true;
1396         return wordInputs.some(inp => inp.value.trim());
1397     }
1398
1399     function clearVisibleSeedInputs() {
1400         els.mnemonic.value = '';
1401         els.mnemonic.classList.remove('valid', 'invalid');
1402         for (const inp of wordInputs) {
1403             inp.value = '';
1404             inp.classList.remove('valid', 'invalid');
1405         }
1406     }

```

```

1407
1408     function clearImportedSeed(options = {}) {
1409         const { silent = false } = options;
1410         if (!importedSeedState) return;
1411         importedSeedState = null;
1412         showingImportedSeed = false;
1413         els.mnemonic.placeholder = 'Paste mnemonic here...';
1414         scheduleRootFingerprintRefresh();
1415         if (!silent) uiStatus('Imported encrypted seed cleared.', '');
1416     }
1417
1418     function setObfuscatedSeedInputs(words) {
1419         const maskWord = '....';
1420         const max = getSeedLen();
1421         els.mnemonic.value = words.map(() => maskWord).join(' ');
1422         for (let i = 0; i < 24; i++) {
1423             wordInputs[i].value = i < words.length && i < max ? maskWord : '';
1424         }
1425         refreshWordEnabled();
1426         styleMnemonicBox();
1427     }
1428
1429     function applyImportedSeed(mnemonic, sourceName) {
1430         const words = normalizeMnemonicWords(mnemonic);
1431         importedSeedState = {
1432             mnemonic: words.join(' '),
1433             sourceName: sourceName || 'encrypted seed file',
1434             kind: 'imported'
1435         };
1436         if (words.length === 12 || words.length === 24) setSeedLen(words.length);
1437         setObfuscatedSeedInputs(words);
1438         els.mnemonic.placeholder = protectedSeedPlaceholder();
1439         uiStatus(protectedSeedStatus(), 'valid');
1440         els.output.value = JSON.stringify({
1441             imported_seed: 'loaded',
1442             source: importedSeedState.sourceName,
1443             word_count: words.length,
1444             hidden_on_screen: true
1445         }, null, 2);
1446         scheduleRootFingerprintRefresh();
1447         // Show derivation path info icon only when path is m/0' (the Arculus default)
1448         function refreshDerivationInfoIcon() {
1449             const icon = document.getElementById('derivationInfoIcon');
1450             if (!icon) return;
1451             const val = (els.derivation.value || '').trim();
1452             icon.style.display = (val === "m/0'") ? '' : 'none';
1453         }
1454         els.derivation.addEventListener('input', refreshDerivationInfoIcon);
1455         // Also re-evaluate whenever the coin selector resets the derivation path
1456         els.coin.addEventListener('change', () => setTimeout(refreshDerivationInfoIcon, 0));
1457         // Show on load – default is Bitcoin with m/0'
1458         window.addEventListener('DOMContentLoaded', refreshDerivationInfoIcon);
1459         refreshDerivationInfoIcon();
1460     }
1461
1462     function applyGeneratedSeed(mnemonic) {
1463         const words = normalizeMnemonicWords(mnemonic);
1464         importedSeedState = {
1465             mnemonic: words.join(' '),
1466             sourceName: 'generated seed',
1467             kind: 'generated'
1468         };

```

```
1469     showingImportedSeed = false;
1470     if (words.length === 12 || words.length === 24) setSeedLen(words.length);
1471     setObfuscatedSeedInputs(words);
1472     els.mnemonic.placeholder = protectedSeedPlaceholder();
1473     scheduleRootFingerprintRefresh();
1474 }
1475
1476 function revealImportedSeed() {
1477     if (!importedSeedState || showingImportedSeed) return;
1478     const words = normalizeMnemonicWords(importedSeedState.mnemonic);
1479     showingImportedSeed = true;
1480     syncBusy = true;
1481     try {
1482         els.mnemonic.value = importedSeedState.mnemonic;
1483         els.mnemonic.placeholder = 'Hold to show seed phrase';
1484         if (words.length === 12 || words.length === 24) setSeedLen(words.length);
1485         setWordBoxesFromMnemonic();
1486         styleMnemonicBox();
1487     } finally {
1488         syncBusy = false;
1489     }
1490 }
1491
1492 function hideImportedSeed() {
1493     if (!importedSeedState || !showingImportedSeed) return;
1494     showingImportedSeed = false;
1495     syncBusy = true;
1496     try {
1497         setObfuscatedSeedInputs(normalizeMnemonicWords(importedSeedState.mnemonic));
1498         els.mnemonic.placeholder = protectedSeedPlaceholder();
1499         uiStatus(protectedSeedStatus(), 'valid');
1500     } finally {
1501         syncBusy = false;
1502     }
1503 }
1504
1505 async function refreshRootFingerprintDisplay() {
1506     const token = ++rootFingerprintRefreshToken;
1507     const mnemonic = getActiveMnemonic();
1508     const words = normalizeMnemonicWords(mnemonic);
1509     if (words.length !== 12 && words.length !== 24) {
1510         setRootFingerprintDisplay('');
1511         return;
1512     }
1513     try {
1514         const analysis = await analyzeMnemonic(mnemonic, els.passphrase.value);
1515         if (token !== rootFingerprintRefreshToken) return;
1516         setRootFingerprintDisplay(analysis.root_fingerprint || '');
1517     } catch {
1518         if (token !== rootFingerprintRefreshToken) return;
1519         setRootFingerprintDisplay('');
1520     }
1521 }
1522
1523 function scheduleRootFingerprintRefresh() {
1524     setRootFingerprintDisplay('');
1525     refreshRootFingerprintDisplay();
1526 }
1527
1528 function styleWordInput(inp) {
1529     const enabled = !inp.disabled;
1530     inp.classList.remove('valid', 'invalid');
```

```

1531     if (importedSeedState && !showingImportedSeed) return;
1532     if (!enabled) return;
1533     const w = normalizeNFKD(inp.value.trim());
1534     if (!w) return;
1535     if (BIP39_SET.has(w)) inp.classList.add('valid');
1536     else inp.classList.add('invalid');
1537 }
1538
1539 function styleMnemonicBox() {
1540     const words = splitWords(els.mnemonic.value);
1541     els.mnemonic.classList.remove('valid', 'invalid');
1542     if (importedSeedState && !showingImportedSeed) return;
1543     if (!words.length) return;
1544     const allValid = words.every(w => BIP39_SET.has(w));
1545     els.mnemonic.classList.add(allValid ? 'valid' : 'invalid');
1546 }
1547
1548 function refreshWordEnabled() {
1549     const max = getSeedLen();
1550     const hiddenProtectedSeed = Boolean(importedSeedState) && !showingImportedSeed;
1551     for (let i = 0; i < 24; i++) {
1552         wordInputs[i].disabled = hiddenProtectedSeed || i >= max;
1553         styleWordInput(wordInputs[i]);
1554         // Hide cells 12-23 when in 12-word mode
1555         const cell = wordInputs[i].closest('.word-cell');
1556         if (cell) cell.style.display = i >= 12 && max === 12 ? 'none' : '';
1557     }
1558 }
1559
1560 function setMnemonicFromWordBoxes() {
1561     const max = getSeedLen();
1562     const ordered = [];
1563     for (let i = 0; i < max; i++) {
1564         const w = normalizeNFKD(wordInputs[i].value.trim());
1565         if (w) ordered.push(w);
1566     }
1567     els.mnemonic.value = ordered.join(' ');
1568     styleMnemonicBox();
1569 }
1570
1571 function setWordBoxesFromMnemonic() {
1572     const words = splitWords(els.mnemonic.value);
1573     const max = getSeedLen();
1574     for (let i = 0; i < max; i++) wordInputs[i].value = words[i] || '';
1575     for (let i = max; i < 24; i++) wordInputs[i].value = '';
1576     for (const inp of wordInputs) styleWordInput(inp);
1577 }
1578
1579 function handleMnemonicInput() {
1580     if (syncBusy) return;
1581     syncBusy = true;
1582     try {
1583         if (importedSeedState && !showingImportedSeed && hasVisibleSeedInput()) clearImportedSeed({ silent: true });
1584         const words = splitWords(els.mnemonic.value);
1585         if (document.activeElement === els.mnemonic) {
1586             if (words.length > 12) setSeedLen(24);
1587             else if (words.length > 0) setSeedLen(12);
1588         }
1589         styleMnemonicBox();
1590         setWordBoxesFromMnemonic();
1591     } finally {
1592         syncBusy = false;

```

```

1593     }
1594     scheduleRootFingerprintRefresh();
1595 }
1596
1597 function handleWordInput() {
1598     if (syncBusy) return;
1599     syncBusy = true;
1600     try {
1601         if (importedSeedState && !showingImportedSeed && hasVisibleSeedInput()) clearImportedSeed({ silent:
1602             for (const inp of wordInputs) styleWordInput(inp);
1603             setMnemonicFromWordBoxes();
1604         } finally {
1605             syncBusy = false;
1606         }
1607         scheduleRootFingerprintRefresh();
1608     }
1609
1610     function detectSeedFormat(wordCount, wordsOk, checksumOk) {
1611         if ((wordCount === 12 || wordCount === 24) && wordsOk && checksumOk) {
1612             return { keystoreType: 'HD wallet mnemonic', seedFormat: 'BIP-39 mnemonic (English)' };
1613         }
1614         if ((wordCount === 12 || wordCount === 24) && wordsOk) {
1615             return { keystoreType: 'Unrecognized or incomplete wallet seed', seedFormat: 'BIP-39-like mnemonic (
1616         }
1617         return { keystoreType: 'Unknown', seedFormat: 'Unknown or unsupported mnemonic format' };
1618     }
1619
1620     async function analyzeMnemonic(mnemonic, passphrase = '') {
1621         const words = normalizeMnemonicWords(mnemonic);
1622         const wc = words.length;
1623         const details = {
1624             word_count: wc,
1625             wordlist_validity: 'Invalid',
1626             entropy_bits: null,
1627             checksum_bits: null,
1628             checksum_match: 'Invalid',
1629             bip39_compliance: 'No',
1630             bip39_seed_512_bit: null,
1631             master_private_key: null,
1632             master_chain_code: null,
1633             root_fingerprint: null,
1634             keystore_type: 'Unknown',
1635             seed_format_detection: 'Unknown or unsupported mnemonic format',
1636             passphrase_warning: null,
1637             message: ''
1638         };
1639
1640         if (wc !== 12 && wc !== 24) {
1641             details.message = 'Only 12-word and 24-word mnemonics are supported.';
1642             return { wordsOk: false, checksumOk: false, ...details };
1643         }
1644
1645         details.checksum_bits = Math.floor(wc / 3);
1646         details.entropy_bits = wc * 11 - details.checksum_bits;
1647
1648         for (const w of words) {
1649             if (!BIP39_SET.has(w)) {
1650                 details.message = 'Mnemonic contains unknown words outside the BIP39 English list.';
1651                 return { wordsOk: false, checksumOk: false, ...details };
1652             }
1653         }
1654         details.wordlist_validity = 'Valid';

```

```

1655
1656     let acc = 0n;
1657     for (const w of words) acc = (acc << 11n) | BigInt(BIP39_INDEX[w]);
1658     const checksumBits = details.checksum_bits;
1659     const entropyBits = details.entropy_bits;
1660     const entropyInt = acc >> BigInt(checksumBits);
1661     const checksumInt = Number(acc & ((1n << BigInt(checksumBits)) - 1n));
1662     const entropy = intToBytes(entropyInt, Math.floor(entropyBits / 8));
1663     const hash = await sha256(entropy);
1664     const expectedChecksum = Number(bytesToBigInt(hash) >> BigInt(256 - checksumBits));
1665     const checksumOk = checksumInt === expectedChecksum;
1666     const seedType = detectSeedFormat(wc, true, checksumOk);
1667
1668     details.checksum_match = checksumOk ? 'Valid' : 'Invalid';
1669     details.bip39_compliance = checksumOk ? 'Yes' : 'No';
1670     details.keystore_type = seedType.keystoreType;
1671     details.seed_format_detection = seedType.seedFormat;
1672     details.passphrase_warning = !passphrase && checksumOk ? 'This seed may require a passphrase' : null;
1673
1674     if (!checksumOk) {
1675         details.message = 'Mnemonic words are valid BIP39 words, but checksum is invalid.';
1676         return { wordsOk: true, checksumOk: false, ...details };
1677     }
1678
1679     const seed = await bip39Seed(mnemonic, passphrase);
1680     const root = await masterFromSeed(seed);
1681
1682     details.bip39_seed_512_bit = bytesToHex(seed);
1683     details.master_private_key = bytesToHex(ser256(root.k));
1684     details.master_chain_code = bytesToHex(root.c);
1685     const rootPub = serPub(pointMul(root.k, G));
1686     details.root_fingerprint = bytesToHex((await hash160(rootPub)).slice(0, 4));
1687     details.message = 'Mnemonic is valid BIP39 (English word list + checksum).';
1688     return { wordsOk: true, checksumOk: true, ...details };
1689 }
1690
1691 async function checkMnemonic(mnemonic, passphrase = '') {
1692     const analysis = await analyzeMnemonic(mnemonic, passphrase);
1693     return { wordsOk: analysis.wordsOk, checksumOk: analysis.checksumOk, message: analysis.message };
1694 }
1695
1696 async function generateRandomMnemonic(wordCount) {
1697     if (wordCount !== 12 && wordCount !== 24) throw new Error('Only 12-word and 24-word mnemonics are supported');
1698     const checksumBits = Math.floor(wordCount / 3);
1699     const entropyBits = wordCount * 11 - checksumBits;
1700     const entropy = crypto.getRandomValues(new Uint8Array(entropyBits / 8));
1701     const hash = await sha256(entropy);
1702     const entropyInt = bytesToBigInt(entropy);
1703     const checksum = bytesToBigInt(hash) >> BigInt(256 - checksumBits);
1704     let bits = (entropyInt << BigInt(checksumBits)) | checksum;
1705     const words = new Array(wordCount);
1706     for (let i = wordCount - 1; i >= 0; i--) {
1707         const idx = Number(bits & 0x7fff);
1708         words[i] = BIP39_WORDS[idx];
1709         bits >>= 11n;
1710     }
1711     const mnemonic = words.join(' ');
1712     const check = await checkMnemonic(mnemonic, '');
1713     if (!(check.wordsOk && check.checksumOk)) throw new Error('Generated mnemonic failed validation.');
1714     return mnemonic;
1715 }
1716
1717 async function textToBytes(s) {

```



```

1717     return new TextEncoder().encode(s);
1718 }
1719
1720 function bytesToText(bytes) {
1721     return new TextDecoder().decode(bytes);
1722 }
1723
1724 function concatBytes(...arrs) {
1725     const len = arrs.reduce((a, b) => a + b.length, 0);
1726     const out = new Uint8Array(len);
1727     let o = 0;
1728     for (const a of arrs) { out.set(a, o); o += a.length; }
1729     return out;
1730 }
1731
1732 async function sha256(bytes) {
1733     const h = await crypto.subtle.digest('SHA-256', bytes);
1734     return new Uint8Array(h);
1735 }
1736
1737 async function hmacSha512(keyBytes, dataBytes) {
1738     const key = await crypto.subtle.importKey('raw', keyBytes, { name: 'HMAC', hash: 'SHA-512' }, false, [
1739     const sig = await crypto.subtle.sign('HMAC', key, dataBytes);
1740     return new Uint8Array(sig);
1741 }
1742
1743 async function pbkdf2Sha512(passwordBytes, saltBytes, iterations, dkLen) {
1744     const key = await crypto.subtle.importKey('raw', passwordBytes, 'PBKDF2', false, ['deriveBits']);
1745     const bits = await crypto.subtle.deriveBits({ name: 'PBKDF2', hash: 'SHA-512', salt: saltBytes, iterations: iterations }, key, dkLen);
1746     return new Uint8Array(bits);
1747 }
1748
1749 async function pbkdf2Sha256(passwordBytes, saltBytes, iterations, dkLen) {
1750     const key = await crypto.subtle.importKey('raw', passwordBytes, 'PBKDF2', false, ['deriveBits']);
1751     const bits = await crypto.subtle.deriveBits({ name: 'PBKDF2', hash: 'SHA-256', salt: saltBytes, iterations: iterations }, key, dkLen);
1752     return new Uint8Array(bits);
1753 }
1754
1755 function bytesToHex(b) { return Array.from(b).map(x => x.toString(16).padStart(2, '0')).join(''); }
1756 function bytesToBigInt(b) { return BigInt('0x' + bytesToHex(b)); }
1757 function intToBytes(n, len) {
1758     const out = new Uint8Array(len);
1759     let x = BigInt(n);
1760     for (let i = len - 1; i >= 0; i--) { out[i] = Number(x & 0xffn); x >>= 8n; }
1761     return out;
1762 }
1763
1764 function keccak256(bytes) {
1765     const mask = (1n << 64n) - 1n;
1766     const rate = 136;
1767     const rot = [
1768         [0, 36, 3, 41, 18],
1769         [1, 44, 10, 45, 2],
1770         [62, 6, 43, 15, 61],
1771         [28, 55, 25, 21, 56],
1772         [27, 20, 39, 8, 14]
1773     ];
1774     const rc = [
1775         0x0000000000000001n, 0x00000000000000802n, 0x80000000000000808an,
1776         0x80000000000000000n, 0x00000000000000808bn, 0x0000000000000001n,
1777         0x800000000000008081n, 0x800000000000008009n, 0x0000000000000008an,
1778         0x0000000000000088n, 0x0000000080008009n, 0x000000008000000an,

```

```

1779      0x0000000008000808bn, 0x8000000000000008bn, 0x80000000000008089n,
1780      0x80000000000008003n, 0x80000000000008002n, 0x800000000000080n,
1781      0x0000000000000800an, 0x8000000008000000an, 0x8000000080008081n,
1782      0x800000000008080n, 0x0000000080000001n, 0x8000000080008008n
1783  ];
1784  const rol64 = (x, n) => n === 0 ? (x & mask) : (((x << BigInt(n)) | (x >> BigInt(64 - n))) & mask);
1785  const state = new Array(25).fill(0n);
1786  const padded = Array.from(bytes);
1787  padded.push(0x01);
1788  while (padded.length % rate !== rate - 1) padded.push(0);
1789  padded.push(0x80);
1790  for (let offset = 0; offset < padded.length; offset += rate) {
1791    for (let i = 0; i < rate / 8; i++) {
1792      let lane = 0n;
1793      for (let j = 0; j < 8; j++) lane |= BigInt(padded[offset + i * 8 + j]) << BigInt(8 * j);
1794      state[i] ^= lane;
1795    }
1796    for (let round = 0; round < 24; round++) {
1797      const c = new Array(5);
1798      const d = new Array(5);
1799      for (let x = 0; x < 5; x++) c[x] = state[x] ^ state[x + 5] ^ state[x + 10] ^ state[x + 15] ^ state[x + 20];
1800      for (let x = 0; x < 5; x++) d[x] = c[(x + 4) % 5] ^ rol64(c[(x + 1) % 5], 1);
1801      for (let x = 0; x < 5; x++) for (let y = 0; y < 5; y++) state[x + 5 * y] = (state[x + 5 * y] ^ d[x] ^ d[y]);
1802      const b = new Array(25).fill(0n);
1803      for (let x = 0; x < 5; x++) for (let y = 0; y < 5; y++) b[y + 5 * ((2 * x + 3 * y) % 5)] = rol64(state[x + 5 * y], 1);
1804      for (let x = 0; x < 5; x++) {
1805        for (let y = 0; y < 5; y++) state[x + 5 * y] = (b[x + 5 * y] ^ ((~b[((x + 1) % 5) + 5 * y]) & b[x + 5 * y]));
1806      }
1807      state[0] = (state[0] ^ rc[round]) & mask;
1808    }
1809  }
1810  const out = new Uint8Array(32);
1811  for (let i = 0; i < 4; i++) for (let j = 0; j < 8; j++) out[i * 8 + j] = Number((state[i] >> BigInt(8 * j)) & 0xff);
1812  return out;
1813 }
1814
1815 function bytesToBase64(bytes) {
1816   let binary = '';
1817   for (const b of bytes) binary += String.fromCharCode(b);
1818   return btoa(binary);
1819 }
1820
1821 function base64ToBytes(text) {
1822   const binary = atob(text);
1823   const out = new Uint8Array(binary.length);
1824   for (let i = 0; i < binary.length; i++) out[i] = binary.charCodeAt(i);
1825   return out;
1826 }
1827
1828 function equalBytes(a, b) {
1829   if (a.length !== b.length) return false;
1830   let diff = 0;
1831   for (let i = 0; i < a.length; i++) diff |= a[i] ^ b[i];
1832   return diff === 0;
1833 }
1834
1835 async function hmacSha256(keyBytes, dataBytes) {
1836   const key = await crypto.subtle.importKey('raw', keyBytes, { name: 'HMAC', hash: 'SHA-256' }, false, ['sign']);
1837   const sig = await crypto.subtle.sign('HMAC', key, dataBytes);
1838   return new Uint8Array(sig);
1839 }
1840

```

```

1841     async function xorStreamCrypt(data, keyBytes, nonceBytes) {
1842         const out = new Uint8Array(data.length);
1843         let offset = 0;
1844         let counter = 0;
1845         while (offset < data.length) {
1846             const block = await sha256(concatBytes(keyBytes, nonceBytes, intToBytes(BigInt(counter), 4)));
1847             const take = Math.min(block.length, data.length - offset);
1848             for (let i = 0; i < take; i++) out[offset + i] = data[offset + i] ^ block[i];
1849             offset += take;
1850             counter += 1;
1851         }
1852         return out;
1853     }
1854
1855     function b64DecodeRequired(value, fieldName, expectedLength = null) {
1856         if (typeof value !== 'string' || !value) throw new Error(`Encrypted seed file is missing ${fieldName}.`);
1857         let bytes;
1858         try {
1859             bytes = base64ToBytes(value);
1860         } catch {
1861             throw new Error(`Encrypted seed file has invalid ${fieldName}.`);
1862         }
1863         if (expectedLength !== null && bytes.length !== expectedLength) {
1864             throw new Error(`Encrypted seed file has invalid ${fieldName} length.`);
1865         }
1866         return bytes;
1867     }
1868
1869     async function arcV2StreamCrypt(data, keyBytes, nonceBytes) {
1870         const out = new Uint8Array(data.length);
1871         let offset = 0;
1872         let counter = 0n;
1873         while (offset < data.length) {
1874             const label = await textToBytes('Arculus ARC v2 stream\u0000');
1875             const block = await hmacSha512(keyBytes, concatBytes(label, nonceBytes, intToBytes(counter, 8)));
1876             const take = Math.min(block.length, data.length - offset);
1877             for (let i = 0; i < take; i++) out[offset + i] = data[offset + i] ^ block[i];
1878             offset += take;
1879             counter += 1n;
1880         }
1881         return out;
1882     }
1883
1884     async function arcV2Keys(password, saltBytes, iterations) {
1885         if (!Number.isInteger(iterations) || iterations < ARC_V2_MIN_KDF_ITERATIONS) {
1886             throw new Error('Encrypted seed file uses too few KDF iterations.');
```

```

1903     String(bundle.version || ''),
1904     String(bundle.created_at || ''),
1905     String(kdf.name || ''),
1906     String(kdf.hash || ''),
1907     String(kdf.iterations || ''),
1908     String(cipher.name || '')
1909 ];
1910 const encoded = [];
1911 for (const part of textParts) encoded.push(await textToBytes(part), sep);
1912 return concatBytes(...encoded, saltBytes, sep, nonceBytes, sep, ciphertext);
1913 }
1914
1915 function normalizeDecryptedMnemonic(payload) {
1916   if (!payload || typeof payload !== 'object') throw new Error('Encrypted seed file contents are invalid');
1917   const mnemonic = normalizeMnemonicWords(payload.mnemonic || '').join(' ');
1918   if (!mnemonic) throw new Error('Encrypted seed file did not contain a mnemonic.');
```

1919 if (payload.word_count !== undefined && Number(payload.word_count) !== normalizeMnemonicWords(mnemonic).length) throw new Error('Encrypted seed file word count does not match mnemonic.');

```

1920   throw new Error('Encrypted seed file word count does not match mnemonic.');
```

1921 }

```

1922   return mnemonic;
1923 }
1924
1925 function mod(a, m) { const r = a % m; return r >= 0n ? r : r + m; }
1926 function modInv(a, m) {
1927   let t = 0n, newT = 1n;
1928   let r = m, newR = mod(a, m);
1929   while (newR !== 0n) {
1930     const q = r / newR;
1931     [t, newT] = [newT, t - q * newT];
1932     [r, newR] = [newR, r - q * newR];
1933   }
1934   if (r > 1n) throw new Error('Not invertible');
1935   return t < 0n ? t + m : t;
1936 }
1937
1938 function pointAdd(p1, p2) {
1939   if (!p1) return p2;
1940   if (!p2) return p1;
1941   const { x: x1, y: y1 } = p1;
1942   const { x: x2, y: y2 } = p2;
1943   if (x1 === x2 && mod(y1 + y2, P) === 0n) return null;
1944   let lam;
1945   if (x1 === x2 && y1 === y2) lam = mod((3n * x1 * x1) * modInv(2n * y1, P), P);
1946   else lam = mod((y2 - y1) * modInv(x2 - x1, P), P);
1947   const x3 = mod(lam * lam - x1 - x2, P);
1948   const y3 = mod(lam * (x1 - x3) - y1, P);
1949   return { x: x3, y: y3 };
1950 }
1951
1952 function pointMul(k, p = G) {
1953   let n = mod(k, N);
1954   if (n === 0n) return null;
1955   let result = null;
1956   let addend = p;
1957   while (n > 0n) {
1958     if (n & 1n) result = pointAdd(result, addend);
1959     addend = pointAdd(addend, addend);
1960     n >>= 1n;
1961   }
1962   return result;
1963 }
1964
```

```

1965     function serPub(p) {
1966         const prefix = (p.y & 1n) ? 0x03 : 0x02;
1967         return concatBytes(Uint8Array.of(prefix), intToBytes(p.x, 32));
1968     }
1969
1970     function ser32(i) { return intToBytes(BigInt(i >>> 0), 4); }
1971     function ser256(i) { return intToBytes(BigInt(i), 32); }
1972
1973     function parsePath(path) {
1974         const out = [];
1975         const parts = path.trim().split('/');
1976         const start = parts[0] === 'm' ? 1 : 0;
1977         for (let i = start; i < parts.length; i++) {
1978             let p = parts[i].trim();
1979             if (!p) continue;
1980             let hard = false;
1981
1982             if (/^[hH]$/.test(p)) { hard = true; p = p.slice(0, -1); }
1983             if (!/^^[d+$/].test(p)) throw new Error(`Invalid derivation index: ${parts[i]}`);
1984             let n = Number(p);
1985             if (n < 0 || n >= HARDENED) throw new Error(`Invalid derivation index: ${parts[i]}`);
1986             if (hard) n |= HARDENED;
1987             out.push(n >>> 0);
1988         }
1989         return out;
1990     }
1991
1992     function normalizePath(path) {
1993         const ints = parsePath(path);
1994         if (!ints.length) return 'm';
1995         return 'm/' + ints.map(i => (i & HARDENED) ? `${i - HARDENED}` : `${i}`).join('/');
1996     }
1997
1998     function purposeToScript(path) {
1999         const ints = parsePath(path);
2000         if (!ints.length) return 'p2pkh';
2001         const p = ints[0] & 0x7fffffff;
2002         if (p === 86) return 'p2tr';
2003         if (p === 84) return 'p2wpkh';
2004         if (p === 49) return 'p2wpkh-p2sh';
2005         return 'p2pkh';
2006     }
2007
2008     function base58encode(bytes, alphabet = B58) {
2009         let x = bytesToBigInt(bytes);
2010         let out = '';
2011         while (x > 0n) {
2012             const mod = x % 58n;
2013             x = x / 58n;
2014             out = alphabet[Number(mod)] + out;
2015         }
2016         let pad = 0;
2017         for (const b of bytes) { if (b === 0) pad++; else break; }
2018         return alphabet[0].repeat(pad) + out;
2019     }
2020
2021     async function base58check(payload, alphabet = B58) {
2022         const c1 = await sha256(payload);
2023         const c2 = await sha256(c1);
2024         return base58encode(concatBytes(payload, c2.slice(0, 4)), alphabet);
2025     }
2026
2027     function bech32Polymod(values) {

```

```

2027     const gen = [0x3b6a57b2, 0x26508e6d, 0x1ea119fa, 0x3d4233dd, 0x2a1462b3];
2028     let chk = 1;
2029     for (const v of values) {
2030         const top = chk >>> 25;
2031         chk = ((chk & 0x1ffffff) << 5) ^ v;
2032         for (let i = 0; i < 5; i++) if ((top >>> i) & 1) chk ^= gen[i];
2033     }
2034     return chk;
2035 }
2036
2037 function bech32HrpExpand(hrp) {
2038     const out = [];
2039     for (const c of hrp) out.push(c.charCodeAt(0) >>> 5);
2040     out.push(0);
2041
2042     for (const c of hrp) out.push(c.charCodeAt(0) & 31);
2043     return out;
2044 }
2045
2046 function bech32EncodeWithConstant(hrp, data, constant) {
2047     const values = bech32HrpExpand(hrp).concat(data).concat([0,0,0,0,0,0]);
2048     const pm = bech32Polymod(values) ^ constant;
2049     const checksum = [];
2050     for (let i = 0; i < 6; i++) checksum.push((pm >>> (5 * (5 - i))) & 31);
2051     return hrp + '1' + data.concat(checksum).map(v => BECH32[v]).join('');
2052 }
2053
2054 function bech32Encode(hrp, data) {
2055     return bech32EncodeWithConstant(hrp, data, 1);
2056 }
2057
2058 function bech32mEncode(hrp, data) {
2059     return bech32EncodeWithConstant(hrp, data, 0x2bc830a3);
2060 }
2061
2062 function convertBits(data, fromBits, toBits, pad = true) {
2063     let acc = 0, bits = 0;
2064     const out = [];
2065     const maxv = (1 << toBits) - 1;
2066     for (const value of data) {
2067         if (value < 0 || (value >> fromBits)) throw new Error('Invalid bits');
2068         acc = (acc << fromBits) | value;
2069         bits += fromBits;
2070         while (bits >= toBits) {
2071             bits -= toBits;
2072             out.push((acc >> bits) & maxv);
2073         }
2074     }
2075     if (pad && bits) out.push((acc << (toBits - bits)) & maxv);
2076     return out;
2077 }
2078
2079 function ripemd160(msg) {
2080     const zl=[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,7,4,13,1,10,6,15,3,12,0,9,5,2,14,11,8,3,10,14,4,9,15,8,14,4,
2081     const zr=[5,14,7,0,9,2,11,4,13,6,15,8,1,10,3,12,6,11,3,7,0,13,5,10,14,15,8,12,4,9,1,2,15,5,1,3,7,14,6,
2082     const sl=[11,14,15,12,5,8,7,9,11,13,14,15,6,7,9,8,7,6,8,13,11,9,7,15,7,12,15,9,11,7,13,12,11,13,6,7,14,
2083     const sr=[8,9,9,11,13,15,15,5,7,7,8,11,14,14,12,6,9,13,15,7,12,8,9,11,7,7,12,7,6,15,13,11,9,7,15,11,8,
2084     const hl=[0x00000000,0x5a827999,0x6ed9eba1,0x8f1bbcdc,0xa953fd4e];
2085     const hr=[0x50a28be6,0x5c4dd124,0x6d703ef3,0x7a6d76e9,0x00000000];
2086     function f(j,x,y,z){ if(j<16)return x^y^z; if(j<32)return (x&y)|(~x&z); if(j<48)return (x|~y)^z; if(j<
2087     function rol(x,n){ return (x<<n)|(x>>> (32-n)); }
2088     const ml = Array.from(msg);
2089     const bitLen = BigInt(ml.length) * 8n;

```

```

2089     ml.push(0x80);
2090     while ((ml.length % 64) !== 56) ml.push(0);
2091     // 64-bit little-endian length field. Do not use JS bitwise shifts here:
2092     // they are 32-bit and would corrupt bytes 4..7.
2093     for (let i = 0n; i < 8n; i++) ml.push(Number((bitLen >> (8n * i)) & 0xffn));
2094     let h0=0x67452301,h1=0xefcdab89,h2=0x98badcfe,h3=0x10325476,h4=0xc3d2e1f0;
2095     for (let i=0;i<ml.length;i+=64){
2096         const X = new Array(16);
2097         for(let j=0;j<16;j++){
2098             const k=i+j*4;
2099             X[j]=(ml[k])|(ml[k+1]<<8)|(ml[k+2]<<16)|(ml[k+3]<<24);
2100         }

2101         let al=h0,bl=h1,cl=h2,dl=h3,el=h4;
2102         let ar=h0,br=h1,cr=h2,dr=h3,er=h4;
2103         for(let j=0;j<80;j++){
2104             const tl=(rol((al + f(j,bl,cl,dl) + X[zl[j]] + hl[Math.floor(j/16)])|0, sl[j]) + el)|0;
2105             al=el; el=dl; dl=rol(cl,10)|0; cl=bl; bl=tl;
2106             const tr=(rol((ar + f(79-j,br,cr,dr) + X[zr[j]] + hr[Math.floor(j/16)])|0, sr[j]) + er)|0;
2107             ar=er; er=dr; dr=rol(cr,10)|0; cr=br; br=tr;
2108         }
2109         const t=(h1 + cl + dr)|0;
2110         h1=(h2 + dl + er)|0;
2111         h2=(h3 + el + ar)|0;
2112         h3=(h4 + al + br)|0;
2113         h4=(h0 + bl + cr)|0;
2114         h0=t;
2115     }
2116     const out = new Uint8Array(20);
2117     const hs=[h0,h1,h2,h3,h4];
2118     for(let i=0;i<5;i++){
2119         out[i*4]=hs[i]&0xff; out[i*4+1]=(hs[i]>>>8)&0xff; out[i*4+2]=(hs[i]>>>16)&0xff; out[i*4+3]=(hs[i]>>>24)&0xff;
2120     }
2121     return out;
2122 }

2123
2124 async function hash160(bytes) {
2125     const s = await sha256(bytes);
2126     return ripemd160(s);
2127 }
2128
2129 async function taggedHash(tag, bytes) {
2130     const tagHash = await sha256(await textToBytes(tag));
2131     return sha256(concatBytes(tagHash, tagHash, bytes));
2132 }
2133
2134 function xOnlyPub(point) {
2135     return intToBytes(point.x, 32);
2136 }
2137
2138 function negatePoint(point) {
2139     return point ? { x: point.x, y: mod(-point.y, P) } : null;
2140 }
2141
2142 async function taprootKeyData(privKey) {
2143     const pub = pointMul(privKey, G);
2144     const evenPriv = (pub.y & 1n) ? mod(N - privKey, N) : privKey;
2145     const evenPub = (pub.y & 1n) ? negatePoint(pub) : pub;
2146     const tweakBytes = await taggedHash('TapTweak', xOnlyPub(evenPub));
2147     const tweak = bytesToBigInt(tweakBytes);
2148     if (tweak >= N) throw new Error('Invalid Taproot tweak');
2149     const outputPub = pointAdd(evenPub, pointMul(tweak, G));
2150     if (!outputPub) throw new Error('Invalid Taproot output key');

```

```

2151     const outputPriv = mod(evenPriv + tweak, N);
2152     if (outputPriv === 0n) throw new Error('Invalid Taproot private key');
2153     return {
2154       internalPrivateKey: evenPriv,
2155       internalPublicKey: xOnlyPub(evenPub),
2156       outputPrivateKey: outputPriv,
2157       outputPublicKey: xOnlyPub(outputPub),
2158       tweak: tweakBytes,
2159       outputParity: (outputPub.y & 1n) ? 1 : 0
2160     };
2161   }
2162
2163   function segwitAddress(hrp, version, witnessProgram) {
2164     const data = [version].concat(convertBits(Array.from(witnessProgram), 8, 5, true));
2165     return version === 0 ? bech32Encode(hrp, data) : bech32mEncode(hrp, data);
2166   }
2167
2168   async function bip39Seed(mnemonic, passphrase) {
2169     // Match Python tool behavior exactly for seed derivation input.
2170     const m = normalizeNFKD((mnemonic ?? '').trim().split(/\s+/).filter(Boolean).join(' '));
2171     const p = normalizeNFKD(passphrase || '');
2172     return pbkdf2Sha512(await textToBytes(m), await textToBytes('mnemonic' + p), 2048, 64);
2173   }
2174
2175   async function masterFromSeed(seed) {
2176     const I = await hmacSha512(await textToBytes('Bitcoin seed'), seed);
2177     const IL = I.slice(0, 32), IR = I.slice(32);
2178     const k = bytesToBigInt(IL);
2179     if (k === 0n || k >= N) throw new Error('Invalid master key');
2180     return { k, c: IR, depth: 0, parentFp: new Uint8Array([0,0,0,0]), childNum: 0 };
2181   }
2182
2183   async function ckdPriv(node, index) {
2184     const idx = index >>> 0;
2185     const parentPub = pointMul(node.k, G);
2186     const data = (idx & HARDENED)
2187       ? concatBytes(Uint8Array.of(0), ser256(node.k), ser32(idx))
2188       : concatBytes(serPub(parentPub), ser32(idx));
2189     const I = await hmacSha512(node.c, data);
2190     const IL = bytesToBigInt(I.slice(0, 32));
2191     const IR = I.slice(32);
2192     const childK = mod(IL + node.k, N);
2193     if (IL >= N || childK === 0n) throw new Error('Invalid child key');
2194     const fp = (await hash160(serPub(parentPub))).slice(0, 4);
2195     return { k: childK, c: IR, depth: node.depth + 1, parentFp: fp, childNum: idx };
2196   }
2197
2198   async function derive(node, path) {
2199     let cur = node;
2200     for (const i of parsePath(path)) cur = await ckdPriv(cur, i);
2201     return cur;
2202   }
2203
2204   async function xprv(node, version) {
2205     const payload = concatBytes(
2206       intToBytes(BigInt(version), 4),
2207       Uint8Array.of(node.depth),
2208       node.parentFp,
2209       ser32(node.childNum),
2210       node.c,
2211       Uint8Array.of(0),
2212       ser256(node.k)

```



```

2213     );
2214     return base58check(payload);
2215 }
2216
2217 async function xpub(node, version) {
2218     const payload = concatBytes(
2219         intToBytes(BigInt(version), 4),
2220         Uint8Array.of(node.depth),
2221
2222         node.parentFp,
2223         ser32(node.childNum),
2224         node.c,
2225         serPub(pointMul(node.k, G))
2226     );
2227     return base58check(payload);
2228 }
2229
2230 async function toWif(k, net) {
2231     if (net.wif === undefined || net.wif === null) throw new Error('WIF is not defined for this coin/network');
2232     return base58check(concatBytes(Uint8Array.of(net.wif), ser256(k), Uint8Array.of(1)));
2233 }
2234
2235 function serPubUncompressed(p) {
2236     return concatBytes(intToBytes(p.x, 32), intToBytes(p.y, 32));
2237 }
2238
2239 async function ethereumChecksumAddress(addressBytes) {
2240     const addressHex = bytesToHex(addressBytes);
2241     const hashHex = bytesToHex(keccak256(await textToBytes(addressHex)));
2242     let checked = '';
2243     for (let i = 0; i < addressHex.length; i++) checked += parseInt(hashHex[i], 16) >= 8 ? addressHex[i].toLowerCase() : addressHex[i].toUpperCase();
2244     return '0x' + checked;
2245 }
2246
2247 async function ethereumAddressFromPrivateKey(privKey) {
2248     const pub = pointMul(privKey, G);
2249     const hash = keccak256(serPubUncompressed(pub));
2250     return ethereumChecksumAddress(hash.slice(-20));
2251 }
2252
2253 async function xrpClassicAddressFromPublicKey(pub) {
2254     const accountId = await hash160(pub);
2255     return base58check(concatBytes(Uint8Array.of(0), accountId), XRP_B58);
2256 }
2257
2258 async function pubToAddress(pub, scriptType, net) {
2259     const pkh = await hash160(pub);
2260     if (scriptType === 'p2pkh') return base58check(concatBytes(Uint8Array.of(net.p2pkhPrefix), pkh));
2261     if (scriptType === 'p2wpkh') {
2262         return segwitAddress(net.hrp, 0, pkh);
2263     }
2264     if (scriptType === 'p2wpkh-p2sh') {
2265         const redeem = concatBytes(Uint8Array.of(0x00, 0x14), pkh);
2266         const h = await hash160(redeem);
2267         return base58check(concatBytes(Uint8Array.of(net.p2shPrefix), h));
2268     }
2269     throw new Error('Unsupported script type');
2270 }
2271
2272 async function deriveOutput() {
2273     const mnemonic = getActiveMnemonic();
2274     const passphrase = els.passphrase.value;
2275     const countRaw = Math.max(1, Number(els.count.value || 5));

```

```

2275     const rangeRaw = (els.rangeInput.value || '').trim();
2276     let startIdx = 0, endIdx = countRaw - 1;
2277     if (rangeRaw) {
2278         const rangeParts = rangeRaw.match(/^(\\d+)\\s*-\\s*(\\d+)$/);
2279         if (rangeParts) {
2280             startIdx = Math.max(0, parseInt(rangeParts[1], 10));
2281             endIdx = Math.max(startIdx, parseInt(rangeParts[2], 10));
2282         }
2283     }
2284     const count = endIdx - startIdx + 1;
2285     const scriptMap = { Auto: 'auto', P2PKH: 'p2pkh', 'P2WPKH-P2SH': 'p2wpkh-p2sh', P2WPKH: 'p2wpkh', P2TR: 'p2tr' };
2286     const requested = scriptMap[els.scriptType.value] || 'p2wpkh';
2287     const coinMap = { Bitcoin: 'bitcoin', Litecoin: 'litecoin', Dogecoin: 'dogecoin', 'Ethereum / ERC-20': 'ethereum' };
2288     const coin = coinMap[els.coin.value] || 'bitcoin';
2289     const net = NETS[coin];
2290     const derivation = normalizePath((els.derivation.value || '').trim() || DEFAULT_ACCOUNT_DERIVATION[coin]);
2291     if (!net) throw new Error(`Unsupported coin: ${coin}`);
2292
2293     const check = await checkMnemonic(mnemonic);
2294     if (!(check.wordsOk && check.checksumOk)) throw new Error(check.message);
2295
2296     const seed = await bip39Seed(mnemonic, passphrase);
2297     const root = await masterFromSeed(seed);
2298     const account = await derive(root, derivation);
2299     const isEthereum = net.addressFamily === 'ethereum';
2300     const isXrp = net.addressFamily === 'xrp';
2301     const st = isEthereum || isXrp ? net.addressFamily : (requested === 'auto' ? purposeToScript(derivation) : requested);
2302     const rootVers = net.p2pkh;
2303     const xVers = net.p2pkh;
2304     const yVers = net['p2wpkh-p2sh'];
2305     const zVers = net.p2wpkh;
2306     const trVers = net.p2tr;
2307     if (!isEthereum && !isXrp && !net[st]) throw new Error('Unsupported script type');
2308
2309     const result = {
2310         coin,
2311         network: net.networkName,
2312         word_count: normalizeMnemonicWords(mnemonic).length,
2313         accounts: [{
2314             derivation,
2315             account_script_type_used: st,
2316             root_xprv: await xprv(root, rootVers.xprv),
2317             root_xpub: await xpub(root, rootVers.xpub)
2318         }]
2319     };
2320     if (st === 'p2wpkh-p2sh') {
2321         result.accounts[0].root_yprv = await xprv(root, yVers.xprv);
2322         result.accounts[0].root_ypub = await xpub(root, yVers.xpub);
2323     } else if (st === 'p2wpkh') {
2324         result.accounts[0].root_zprv = await xprv(root, zVers.xprv);
2325         result.accounts[0].root_zpub = await xpub(root, zVers.xpub);
2326     } else if (st === 'p2tr' && trVers) {
2327         result.accounts[0].root_trprv = await xprv(root, trVers.xprv);
2328         result.accounts[0].root_trpub = await xpub(root, trVers.xpub);
2329     }
2330     result.accounts[0].account_xprv = await xprv(account, xVers.xprv);
2331     result.accounts[0].account_xpub = await xpub(account, xVers.xpub);
2332     if (st === 'p2wpkh-p2sh') {
2333         result.accounts[0].account_yprv = await xprv(account, yVers.xprv);
2334         result.accounts[0].account_ypub = await xpub(account, yVers.xpub);
2335     } else if (st === 'p2wpkh') {
2336         result.accounts[0].account_zprv = await xprv(account, zVers.xprv);

```

```

2337     result.accounts[0].account_zpub = await xpub(account, zVers.xpub);
2338   } else if (st === 'p2tr' && trVers) {
2339     result.accounts[0].account_trprv = await xprv(account, trVers.xprv);
2340     result.accounts[0].account_trpub = await xpub(account, trVers.xpub);
2341   }
2342   result.accounts[0].receiving = [];
2343   result.accounts[0].change = [];
2344
2345   const totalAddresses = count * 2;
2346   let addressesDerived = 0;
2347   for (const [branch, key] of [[0, 'receiving'], [1, 'change']]) {
2348     const branchNode = await ckdPriv(account, branch);
2349     for (let i = 0; i < count; i++) {
2350       const child = await ckdPriv(branchNode, startIdx + i);
2351       addressesDerived++;
2352       setProgress(10 + Math.round((addressesDerived / totalAddresses) * 80));
2353       if (addressesDerived % 3 === 0) await new Promise(r => setTimeout(r, 0));
2354       const pub = serPub(pointMul(child.k, G));
2355       const item = {
2356         path: `${derivation}/${branch}/${startIdx + i}`,
2357         address: st === 'p2tr' || isEthereum || isXrp ? null : await pubToAddress(pub, st, net),
2358         public_key_hex: bytesToHex(pub),
2359         private_key_hex: bytesToHex(ser256(child.k))
2360       };
2361       if (isEthereum) {
2362         item.address = await ethereumAddressFromPrivateKey(child.k);
2363         item.ethereum_public_key_hex = bytesToHex(serPubUncompressed(pointMul(child.k, G)));
2364         item.erc20_address = item.address;
2365         item.erc20_note = 'ERC-20 tokens on Ethereum use this same account address.';
2366       } else if (isXrp) {
2367         item.address = await xrpClassicAddressFromPublicKey(pub);
2368         item.xrp_classic_address = item.address;
2369         item.xrp_note = 'XRP Ledger classic address. Destination tags, when required by an exchange or o
2370       } else {
2371         item.private_key_wif = await toWif(child.k, net);
2372       }
2373       if (st === 'p2tr') {
2374         const taproot = await taprootKeyData(child.k);
2375         item.address = segwitAddress(net.hrp, 1, taproot.outputPublicKey);
2376         item.taproot_internal_public_key_hex = bytesToHex(taproot.internalPublicKey);
2377         item.taproot_internal_private_key_hex = bytesToHex(ser256(taproot.internalPrivateKey));
2378         item.taproot_internal_private_key_wif = await toWif(taproot.internalPrivateKey, net);
2379         item.taproot_tweak_hex = bytesToHex(taproot.tweak);
2380         item.taproot_output_public_key_hex = bytesToHex(taproot.outputPublicKey);
2381         item.taproot_output_private_key_hex = bytesToHex(ser256(taproot.outputPrivateKey));
2382         item.taproot_output_private_key_wif = await toWif(taproot.outputPrivateKey, net);
2383         item.taproot_output_key_parity = taproot.outputParity;
2384       }
2385       result.accounts[0][key].push(item);
2386     }
2387   }
2388
2389   return result;
2390 }
2391
2392 function flattenDerivedRows(data) {
2393   const rows = [];
2394   for (const account of data.accounts || []) {
2395     const EXCLUDE_FROM_TABLE = new Set([...AK_KEY_ORDER, 'network', 'word_count', 'derivation', 'account
2396     const accountFields = {};
2397     for (const [key, value] of Object.entries(account)) {
2398       if (key !== 'receiving' && key !== 'change' && !EXCLUDE_FROM_TABLE.has(key)) accountFields[key] =

```

```

2399     }
2400     for (const branch of ['receiving', 'change']) {
2401         for (const item of account[branch] || []) {
2402             rows.push({
2403                 branch,
2404                 ...accountFields,
2405                 ...item
2406             });
2407         }
2408     }
2409 }
2410 return rows;
2411 }
2412
2413 function csvEscape(value) {
2414     const text = value === null || value === undefined ? '' : String(value);
2415     return /[",\r\n]/.test(text) ? `"${text.replace(/"/g, '""')}"` : text;
2416 }
2417
2418 function derivedToCsv(data) {
2419     const rows = flattenDerivedRows(data);
2420     if (!rows.length) return '';
2421     const headers = Array.from(rows.reduce((set, row) => {
2422         Object.keys(row).forEach(key => set.add(key));
2423         return set;
2424     }, new Set()));
2425     return [
2426         headers.map(csvEscape).join(','),
2427         ...rows.map(row => headers.map(header => csvEscape(row[header])).join(','))
2428     ].join('\n') + '\n';
2429 }
2430
2431 function textLabel(key) {
2432     return key.replace(/_/g, ' ');
2433 }
2434
2435 function appendTextFields(lines, obj, exclude = []) {
2436     const excluded = new Set(exclude);
2437     for (const [key, value] of Object.entries(obj || {})) {
2438         if (excluded.has(key) || value === null || value === undefined) continue;
2439         lines.push(`${textLabel(key)}: ${value}`);
2440     }
2441 }
2442
2443 function derivedToText(data) {
2444     const lines = [
2445         'Arculus Derived Keys and Addresses',
2446         '',
2447         `coin: ${data.coin || ''}`,
2448         `network: ${data.network || ''}`,
2449         `word count: ${data.word_count || ''}`
2450     ];
2451     for (const [accountIndex, account] of (data.accounts || []).entries()) {
2452         lines.push(`Account ${accountIndex + 1}`);
2453         appendTextFields(lines, account, ['receiving', 'change']);
2454         for (const branch of ['receiving', 'change']) {
2455             lines.push(` ${branch.charAt(0).toUpperCase()}${branch.slice(1)} Addresses`);
2456             const items = account[branch] || [];
2457             if (!items.length) {
2458                 lines.push('No addresses derived.');
```

```

2461     for (const [itemIndex, item] of items.entries()) {
2462         lines.push('', `${branch} #${itemIndex}`);
2463         appendTextFields(lines, item);
2464     }
2465 }
2466 }
2467 return lines.join('\n') + '\n';
2468 }
2469
2470 function exportPayload(data, format) {
2471     if (format === 'csv') {
2472         return {
2473             text: derivedToCsv(data),
2474             type: 'text/csv;charset=utf-8',
2475             filename: 'Arculus_Derived_Keys_Addresses.csv'
2476         };
2477     }
2478     if (format === 'txt') {
2479         return {
2480             text: derivedToText(data),
2481             type: 'text/plain;charset=utf-8',
2482             filename: 'Arculus_Derived_Keys_Addresses.txt'
2483         };
2484     }
2485     return {
2486         text: JSON.stringify(data, null, 2) + '\n',
2487         type: 'application/json;charset=utf-8',
2488         filename: 'Arculus_Derived_Keys_Addresses.json'
2489     };
2490 }
2491
2492 async function exportDump() {
2493     if (!lastDerivedOutput) {
2494         uiStatus('Nothing to export. Derive keys and addresses first.');
```

[illegible]

[illegible]

[illegible]


```

2709     if (!lastDerivedOutput) {
2710         uiStatus('Nothing to export. Derive keys and addresses first.');
```

2711 return;

2712 }

2713 const format = els.exportFormat.value.toLowerCase();

2714 if (format === 'pdf') {

2715 exportDump().catch(function(e) {

2716 uiStatus('Error: ' + e.message, 'invalid');

2717 });

2718 return;

2719 }

2720 const payload = exportPayload(lastDerivedOutput, format);

2721 const blob = new Blob([payload.text], { type: payload.type });

2722 const a = document.createElement('a');

2723 a.href = URL.createObjectURL(blob);

2724 a.download = payload.filename;

2725 document.body.appendChild(a);

2726 a.click();

2727 URL.revokeObjectURL(a.href);

2728 a.remove();

2729 uiStatus(`Exported derived keys/addresses as \${format.toUpperCase()}.`);

2730 }

2731

2732 async function encrypt_v2(mnemonic, password) {

2733 const words = normalizeMnemonicWords(mnemonic);

2734 if (![12, 24].includes(words.length)) throw new Error('Only 12-word and 24-word mnemonics can be encrypted');

2735 const salt = crypto.getRandomValues(new Uint8Array(ARC_V2_SALT_BYTES));

2736 const nonce = crypto.getRandomValues(new Uint8Array(ARC_V2_NONCE_BYTES));

2737 const createdAt = new Date().toISOString();

2738 const bundle = {

2739 magic: ARC_MAGIC,

2740 format: ARC_V2_FORMAT,

2741 version: 2,

2742 created_at: createdAt,

2743 kdf: { name: 'PBKDF2', hash: 'SHA-512', iterations: ARC_V2_KDF_ITERATIONS, salt_b64: bytesToBase64(salt) },

2744 cipher: { name: 'HMAC-SHA512-CTR', nonce_b64: bytesToBase64(nonce) }

2745 };

2746 const payload = {

2747 mnemonic: words.join(' '),

2748 word_count: words.length,

2749 created_at: createdAt

2750 };

2751 const plaintext = await textToBytes(JSON.stringify(payload));

2752 const { encKey, macKey } = await arcV2Keys(password, salt, ARC_V2_KDF_ITERATIONS);

2753 const ciphertext = await arcV2StreamCrypt(plaintext, encKey, nonce);

2754 bundle.ciphertext_b64 = bytesToBase64(ciphertext);

2755 const mac = await hmacSha512(macKey, await arcV2MacData(bundle, salt, nonce, ciphertext));

2756 bundle.mac_b64 = bytesToBase64(mac);

2757 return bundle;

2758 }

2759

2760 async function decryptV2(bundle, password) {

2761 if (bundle.magic !== ARC_MAGIC || bundle.version !== 2 || bundle.format !== ARC_V2_FORMAT) {

2762 throw new Error('Unsupported encrypted seed file format.');

2763 }

2764 if (!bundle.kdf || !bundle.cipher) throw new Error('Encrypted seed file is malformed.');

2765 if (bundle.kdf.name !== 'PBKDF2' || bundle.kdf.hash !== 'SHA-512') throw new Error('Unsupported encryption method.');

2766 if (bundle.cipher.name !== 'HMAC-SHA512-CTR') throw new Error('Unsupported encrypted seed file cipher.');

2767 const iterations = Number(bundle.kdf.iterations);

2768 const salt = b64DecodeRequired(bundle.kdf.salt_b64, 'salt_b64', ARC_V2_SALT_BYTES);

2769 const nonce = b64DecodeRequired(bundle.cipher.nonce_b64, 'nonce_b64', ARC_V2_NONCE_BYTES);

2770 const ciphertext = b64DecodeRequired(bundle.ciphertext_b64, 'ciphertext_b64');

```

2771     const expectedMac = b64DecodeRequired(bundle.mac_b64, 'mac_b64', 64);
2772     const { encKey, macKey } = await arcV2Keys(password, salt, iterations);
2773     const actualMac = await hmacSha512(macKey, await arcV2MacData(bundle, salt, nonce, ciphertext));
2774     if (!equalBytes(actualMac, expectedMac)) {
2775         throw new Error('Unable to decrypt seed file. The password may be incorrect or the file may be corrupt');
2776     }
2777     const decrypted = await arcV2StreamCrypt(ciphertext, encKey, nonce);
2778     const payload = JSON.parse(new TextDecoder().decode(decrypted));
2779     return normalizeDecryptedMnemonic(payload);
2780 }
2781
2782 async function decryptV1(bundle, password) {
2783     if (bundle.format === 'arculus-encrypted-seed-v2' || bundle.format === 'arculus-encrypted-seed-python') {
2784         const salt = base64ToBytes(bundle.kdf?.salt_b64 || '');
2785         const nonce = base64ToBytes(bundle.cipher?.nonce_b64 || '');
2786         const ciphertext = base64ToBytes(bundle.ciphertext_b64 || '');
2787         const expectedMac = base64ToBytes(bundle.mac_b64 || '');
2788         const iterations = Number(bundle.kdf?.iterations || 250000);
2789         const passwordBytes = await textToBytes(normalizeNFKD(password));
2790         const derived = await pbkdf2Sha256(passwordBytes, salt, iterations, 64);
2791         const encKey = derived.slice(0, 32);
2792         const macKey = derived.slice(32);
2793         const actualMac = await hmacSha256(macKey, concatBytes(nonce, ciphertext));
2794         if (!equalBytes(actualMac, expectedMac)) {
2795             throw new Error('Unable to decrypt seed file. The password may be incorrect or the file may be corrupt');
2796         }
2797         const decrypted = await xorStreamCrypt(ciphertext, encKey, nonce);
2798         const payload = JSON.parse(new TextDecoder().decode(decrypted));
2799         return normalizeDecryptedMnemonic(payload);
2800     }
2801     if (bundle.format === 'arculus-encrypted-seed-v1') {
2802         const salt = base64ToBytes(bundle.kdf?.salt_b64 || '');
2803         const iv = base64ToBytes(bundle.cipher?.iv_b64 || '');
2804         const ciphertext = base64ToBytes(bundle.ciphertext_b64 || '');
2805         const iterations = Number(bundle.kdf?.iterations || 250000);
2806         const passwordBytes = await textToBytes(normalizeNFKD(password));
2807         const keyMaterial = await crypto.subtle.importKey('raw', passwordBytes, 'PBKDF2', false, ['deriveKey', 'deriveBits']);
2808         const key = await crypto.subtle.deriveKey(
2809             { name: 'PBKDF2', hash: 'SHA-256', salt, iterations },
2810             keyMaterial,
2811             { name: 'AES-GCM', length: 256 },
2812             false,
2813             ['decrypt']
2814         );
2815         let decrypted;
2816         try {
2817             decrypted = await crypto.subtle.decrypt({ name: 'AES-GCM', iv }, key, ciphertext);
2818         } catch {
2819             throw new Error('Unable to decrypt seed file. The password may be incorrect or the file may be corrupt');
2820         }
2821         const payload = JSON.parse(new TextDecoder().decode(decrypted));
2822         return normalizeDecryptedMnemonic(payload);
2823     }
2824     throw new Error('Unsupported encrypted seed file format. ');
2825 }
2826
2827 async function decrypt(bundle, password) {
2828     if (!bundle || typeof bundle !== 'object') throw new Error('Unsupported encrypted seed file format. ');
2829     if (bundle.magic === ARC_MAGIC || bundle.version === 2) return decryptV2(bundle, password);
2830     return decryptV1(bundle, password);
2831 }
2832

```

```
2833     async function encryptSeedBundle(mnemonic, password) {
2834         return encrypt_v2(mnemonic, password);
2835     }
2836
2837     async function decryptSeedBundle(bundle, password) {
2838         return decrypt(bundle, password);
2839     }
2840
2841     async function serializeSeedBundle(bundle) {
2842         const json = JSON.stringify(bundle);
2843         return `${ARC_ARMOR_HEADER}\n${bytesToBase64(await textToBytes(json))}\n`;
2844     }
2845
2846     function parseSeedBundle(text) {
2847         const trimmed = text.trim();
2848         if (trimmed.startsWith(ARC_ARMOR_HEADER)) {
2849             const encoded = trimmed.split(/\r?\n/).slice(1).join('').trim();
2850             try {
2851                 return JSON.parse(bytesToText(base64ToBytes(encoded)));
2852             } catch {
2853                 throw new Error('Selected encrypted seed file armor is invalid.');
```

```
2854             }
2855         }
2856         try {
2857             return JSON.parse(text);
2858         } catch {
2859             throw new Error('Selected file is not a supported encrypted seed file.');
```

```
2860         }
2861     }
2862
2863     async function handleEncryptExportSeed() {
2864         const mnemonic = getActiveMnemonic();
2865         const check = await checkMnemonic(mnemonic, els.passphrase.value);
2866         if (!(check.wordsOk && check.checksumOk)) throw new Error('Load a valid mnemonic before exporting an e
2867         const password = window.prompt('Enter a password to encrypt this seed export file:');
2868         if (password === null) return;
2869         if (!password) throw new Error('Encryption password cannot be empty.');
```

```
2870         const confirmPassword = window.prompt('Re-enter the password to confirm:');
2871         if (confirmPassword === null) return;
2872         if (password !== confirmPassword) throw new Error('Passwords did not match.');
```

```
2873         const bundle = await encryptSeedBundle(mnemonic, password);
2874         const blob = new Blob([await serializeSeedBundle(bundle)], { type: 'text/plain;charset=utf-8' });
2875         const a = document.createElement('a');
2876         a.href = URL.createObjectURL(blob);
2877         a.download = 'Arculus_Encrypted_Seed.arc';
2878         document.body.appendChild(a);
2879         a.click();
2880         URL.revokeObjectURL(a.href);
2881
2882         a.remove();
2882         uiStatus('Encrypted seed file exported.', 'valid');
```

```
2883     }
2884
2885     async function handleImportSeedFile(file) {
2886         if (!file) return;
2887         const password = window.prompt('Enter the password for this encrypted seed file:');
2888         if (password === null) return;
2889         if (!password) throw new Error('Import password cannot be empty.');
```

```
2890         const text = await file.text();
2891         const bundle = parseSeedBundle(text);
2892         const mnemonic = await decryptSeedBundle(bundle, password);
2893         const check = await checkMnemonic(mnemonic, els.passphrase.value);
2894         if (!(check.wordsOk && check.checksumOk)) throw new Error('Imported file decrypted successfully, but t
```

```
2895     applyImportedSeed(mnemonic, file.name);
2896 }
2897
2898 async function copyTextToClipboard(text) {
2899     if (navigator.clipboard?.writeText) {
2900         await navigator.clipboard.writeText(text);
2901         return;
2902     }
2903     const temp = document.createElement('textarea');
2904     temp.value = text;
2905     temp.setAttribute('readonly', '');
2906     temp.style.position = 'fixed';
2907     temp.style.opacity = '0';
2908     document.body.appendChild(temp);
2909     temp.select();
2910     document.execCommand('copy');
2911     temp.remove();
2912 }
2913
2914 async function handleCopySeed() {
2915     const mnemonic = getActiveMnemonic();
2916     const words = normalizeMnemonicWords(mnemonic);
2917     if (!words.length) throw new Error('No seed phrase is currently loaded.');
```

const ok = window.confirm('Copying a seed phrase to the clipboard is not recommended because other applications may steal it.');

```
2918     if (!ok) {
2919         uiStatus('Copy seed canceled.', '');
2920         return;
2921     }
2922     await copyTextToClipboard(words.join(' '));
2923     els.status.className = 'valid';
2924     els.status.textContent = 'Seed phrase copied to clipboard.';
2925     const countdownSpan = document.createElement('span');
2926     countdownSpan.className = 'clipboard-countdown';
2927     els.status.appendChild(countdownSpan);
2928     startClipboardCountdown(countdownSpan);
2929 }
2930
2931 async function handleGenerateSeed() {
2932     const wordCount = getSeedLen();
2933     const mnemonic = await generateRandomMnemonic(wordCount);
2934     syncBusy = true;
2935     try {
2936         applyGeneratedSeed(mnemonic);
2937     } finally {
2938         syncBusy = false;
2939     }
2940
2941     uiStatus(`Generated random ${wordCount}-word mnemonic. Seed phrase hidden; hold Show Seed to reveal.`);
2942 }
2943
2944 function handleClearAll() {
2945     clearImportedSeed({ silent: true });
2946     syncBusy = true;
2947     try {
2948         els.mnemonic.value = '';
2949         for (const inp of wordInputs) {
2950             inp.value = '';
2951             inp.classList.remove('valid', 'invalid');
2952         }
2953         els.mnemonic.classList.remove('valid', 'invalid');
2954         els.passphrase.value = '';
2955         els.output.value = '';
2956         setRootFingerprintDisplay('');
```

```

2957     } finally {
2958         syncBusy = false;
2959     }
2960     styleMnemonicBox();
2961     resetOutputToRaw();
2962     lastDerivedOutput = null;
2963     uiStatus('All fields cleared.', '');
2964 }
2965
2966 function buildWordInputs() {
2967     for (let i = 0; i < 24; i++) {
2968         const cell = document.createElement('div');
2969         cell.className = 'word-cell';
2970         const num = document.createElement('div');
2971         num.className = 'word-num';
2972         num.textContent = `${i + 1}.`;
2973         const input = document.createElement('input');
2974         input.type = 'text';
2975         input.spellcheck = false;
2976         input.autocomplete = 'off';
2977         input.addEventListener('input', handleWordInput);
2978         input.addEventListener('blur', handleWordInput);
2979         cell.appendChild(num);
2980         cell.appendChild(input);
2981         els.wordsGrid.appendChild(cell);
2982         wordInputs.push(input);
2983     }
2984 }
2985
2986 ■ function bindUI() {
2987 ■     els.mnemonic.addEventListener('input', handleMnemonicInput);
2988     document.querySelectorAll('input[name="seedLen"]').forEach(r => r.addEventListener('change', () => {
2989         refreshWordEnabled();
2990         if (importedSeedState) return;
2991         handleWordInput();
2992     }));
2993     els.coin.addEventListener('change', () => {
2994         const coinMap = { Bitcoin: 'bitcoin', Litecoin: 'litecoin', Dogecoin: 'dogecoin', 'Ethereum / ERC-20': 'ethereum' };
2995         const key = coinMap[els.coin.value] || 'bitcoin';
2996         els.derivation.value = DEFAULT_ACCOUNT_DERIVATION[key];
2997         if (key === 'dogecoin') {
2998             els.scriptType.value = 'P2PKH';
2999         } else if (key === 'ethereum' || key === 'xrp') {
3000             els.scriptType.value = 'Auto';
3001         } else {
3002             els.scriptType.value = 'P2WPKH';
3003         }
3004     });
3005 ■     els.passphrase.addEventListener('input', scheduleRootFingerprintRefresh);
3006     // Passphrase visibility toggle
3007     document.getElementById('passphraseToggle').addEventListener('click', function() {
3008         const inp = els.passphrase;
3009         const btn = document.getElementById('passphraseToggle');
3010         if (inp.type === 'password') {
3011             inp.type = 'text';
3012             btn.setAttribute('aria-label', 'Hide passphrase');
3013             btn.innerHTML = '<svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"><path d="M12 15c1.66 0 3-1.34 3-3s-1.34-3-3-3-3 1.34-3 3 1.34 3 3 3z"/></svg>';
3014         } else {
3015             inp.type = 'password';
3016             btn.setAttribute('aria-label', 'Show passphrase');
3017             btn.innerHTML = '<svg xmlns="http://www.w3.org/2000/svg" width="18" height="18" viewBox="0 0 24 24"><path d="M12 15c1.66 0 3-1.34 3-3s-1.34-3-3-3-3 1.34-3 3 1.34 3 3 3z"/></svg>';
3018         }
3019     });

```

```

3019     });
3020     ■   els.settingsBtn.addEventListener('click', () => {
3021     ■       if (typeof els.settingsDialog.showModal === 'function') {
3022     ■           els.settingsDialog.showModal();
3023     ■       } else {
3024     ■           els.settingsDialog.setAttribute('open', '');
3025     ■       }
3026     ■   });
3027     ■   els.settingsCloseBtn.addEventListener('click', () => els.settingsDialog.close());
3028     ■   els.settingsDialog.addEventListener('click', event => {
3029     ■       if (event.target === els.settingsDialog) els.settingsDialog.close();
3030     ■   });
3031     ■   document.querySelectorAll('input[name="themeMode"]').forEach(r => r.addEventListener('change', () =>
3032     ■   window.addEventListener('online', refreshNetworkStatus);
3033     ■   window.addEventListener('offline', refreshNetworkStatus);
3034     ■
3035     ■   els.validateBtn.addEventListener('click', async () => {
3036     ■       try {
3037     ■           const res = await analyzeMnemonic(getActiveMnemonic(), els.passphrase.value);
3038     ■           setRootFingerprintDisplay(res.root_fingerprint || '');
3039     ■           uiStatus(res.message, res.wordsOk && res.checksumOk ? 'valid' : 'invalid');
3040     ■           resetOutputToRaw();
3041     ■           els.output.value = JSON.stringify({
3042     ■               validation: res.wordsOk && res.checksumOk ? 'ok' : 'failed',
3043     ■               word_count: res.word_count,
3044     ■               wordlist_validity: res.wordlist_validity,
3045     ■               entropy_bits: res.entropy_bits,
3046     ■               checksum_bits: res.checksum_bits,
3047     ■               checksum_match: res.checksum_match,
3048     ■               bip39_compliance: res.bip39_compliance,
3049     ■               bip39_seed_512_bit: res.bip39_seed_512_bit,
3050     ■               master_private_key: res.master_private_key,
3051     ■               master_chain_code: res.master_chain_code,
3052     ■               root_fingerprint: res.root_fingerprint,
3053     ■               keystore_type: res.keystore_type,
3054     ■               seed_format_detection: res.seed_format_detection,
3055     ■               passphrase_warning: res.passphrase_warning,
3056     ■               message: res.message
3057     ■           }, null, 2);
3058     ■       } catch (e) {
3059     ■           uiStatus(`Error: ${e.message}`, 'invalid');
3060     ■           els.output.value = JSON.stringify({ error: e.message }, null, 2);
3061     ■       }
3062     ■   });
3063
3064     // Range input – blank the count box while a range is active, restore when cleared.
3065     els.rangeInput.addEventListener('input', () => {
3066     ■       const hasRange = els.rangeInput.value.trim() !== '';
3067     ■       els.count.value = hasRange ? '' : (els.count._saved || 5);
3068     ■       els.count.disabled = hasRange;
3069     ■       els.count.style.opacity = hasRange ? '0.4' : '';
3070     ■   });
3071     // Persist the last manual count value so we can restore it when range is cleared.
3072     els.count.addEventListener('input', () => {
3073     ■       if (!els.rangeInput.value.trim()) els.count._saved = els.count.value;
3074     ■   });
3075
3076     // Auto-re-derive – when the user changes coin, script type, derivation path,
3077     // address count, or range AFTER a successful derive has already run, re-run
3078     // automatically after a short debounce so the output stays in sync with the
3079     // settings without requiring a manual button press.
3080     let _autoderiveTimer = null;

```

```

3081     function scheduleAutoderive() {
3082         if (!lastDerivedOutput) return; // nothing derived yet – don't fire
3083         if (!document.getElementById('autoderiveToggle').checked) return; // disabled in settings
3084         clearTimeout(_autoderiveTimer);
3085         _autoderiveTimer = setTimeout(async () => {
3086             try {
3087                 uiStatus('Re-deriving...');
3088                 setProgress(5);
3089                 const data = await deriveOutput();
3090                 setProgress(95);
3091                 lastDerivedOutput = data;
3092                 els.output.value = JSON.stringify(data, null, 2);
3093                 setProgress(100);
3094                 setTimeout(hideProgress, 400);
3095                 showDerivedTable(data);
3096                 uiStatus('Derivation complete.', 'valid');
3097             } catch (e) {
3098                 hideProgress();
3099                 uiStatus(`Error: ${e.message}`, 'invalid');
3100             }
3101         }, 600);
3102     }
3103     els.coin.addEventListener('change', scheduleAutoderive);
3104     els.scriptType.addEventListener('change', scheduleAutoderive);
3105     els.derivation.addEventListener('input', scheduleAutoderive);
3106     els.count.addEventListener('input', scheduleAutoderive);
3107     els.rangeInput.addEventListener('input', scheduleAutoderive);
3108
3109     // Advanced section – expand/collapse toggle
3110     document.getElementById('advancedToggleBtn').addEventListener('click', () => {
3111         const section = document.getElementById('advancedSection');
3112         const chevron = document.getElementById('advancedChevron');
3113         const btn = document.getElementById('advancedToggleBtn');
3114         const expanded = section.style.display !== 'none';
3115         section.style.display = expanded ? 'none' : '';
3116         chevron.style.transform = expanded ? '' : 'rotate(90deg)';
3117         btn.setAttribute('aria-expanded', String(!expanded));
3118     });
3119
3120     els.deriveBtn.addEventListener('click', async () => {
3121         try {
3122             uiStatus('Deriving...');
3123             setProgress(5);
3124             const data = await deriveOutput();
3125             setProgress(95);
3126             lastDerivedOutput = data;
3127             els.output.value = JSON.stringify(data, null, 2);
3128             setProgress(100);
3129             setTimeout(hideProgress, 400);
3130             showDerivedTable(data);
3131             uiStatus('Derivation complete.', 'valid');
3132         } catch (e) {
3133             hideProgress();
3134             uiStatus(`Error: ${e.message}`, 'invalid');
3135             els.output.value = JSON.stringify({ error: e.message }, null, 2);
3136         }
3137     });
3138
3139     els.exportBtn.addEventListener('click', exportOutput);
3140     els.encryptExportBtn.addEventListener('click', async () => {
3141         try {
3142             await handleEncryptExportSeed();

```



```

3143         } catch (e) {
3144             uiStatus(`Error: ${e.message}`, 'invalid');
3145             els.output.value = JSON.stringify({ error: e.message }, null, 2);
3146         }
3147     });
3148     els.importSeedBtn.addEventListener('click', () => {
3149         els.importSeedFile.value = '';
3150         els.importSeedFile.click();
3151     });
3152     els.copySeedBtn.addEventListener('click', async () => {
3153         try {
3154             await handleCopySeed();
3155         } catch (e) {
3156             uiStatus(`Error: ${e.message}`, 'invalid');
3157             els.output.value = JSON.stringify({ error: e.message }, null, 2);
3158         }
3159     });
3160     els.generateSeedBtn.addEventListener('click', async () => {
3161         try {
3162             await handleGenerateSeed();
3163         } catch (e) {
3164             uiStatus(`Error: ${e.message}`, 'invalid');
3165             els.output.value = JSON.stringify({ error: e.message }, null, 2);
3166         }
3167     });
3168     els.clearAllBtn.addEventListener('click', () => handleClearAll());
3169     els.importSeedFile.addEventListener('change', async () => {
3170         try {
3171             await handleImportSeedFile(els.importSeedFile.files?.[0]);
3172         } catch (e) {
3173             uiStatus(`Error: ${e.message}`, 'invalid');
3174             els.output.value = JSON.stringify({ error: e.message }, null, 2);
3175         } finally {
3176             els.importSeedFile.value = '';
3177         }
3178     });
3179     const stopShowSeed = () => hideImportedSeed();
3180     els.showSeedBtn.addEventListener('mousedown', () => revealImportedSeed());

3181     els.showSeedBtn.addEventListener('mouseup', stopShowSeed);
3182     els.showSeedBtn.addEventListener('mouseleave', stopShowSeed);
3183     els.showSeedBtn.addEventListener('touchstart', e => {
3184         e.preventDefault();
3185         revealImportedSeed();
3186     }, { passive: false });
3187     els.showSeedBtn.addEventListener('touchend', stopShowSeed);
3188     els.showSeedBtn.addEventListener('touchcancel', stopShowSeed);
3189     scheduleRootFingerprintRefresh();
3190     // Output tab switching – exclude the expand button from tab logic
3191     document.querySelectorAll('.output-tab:not(#expandOutputBtn)').forEach(function(btn) {
3192         btn.addEventListener('click', function() { setOutputTab(btn.dataset.tab); });
3193     });
3194
3195     // Expand / minimize output panel
3196     document.getElementById('expandOutputBtn').addEventListener('click', function() {
3197         const isExpanded = document.body.classList.toggle('output-expanded');
3198         this.textContent = isExpanded ? '■ Minimize' : '■ Expand';
3199         this.title = isExpanded ? 'Minimize output panel' : 'Expand output panel';
3200         // Prevent page scroll while expanded
3201         document.body.style.overflow = isExpanded ? 'hidden' : '';
3202     });
3203     // Click backdrop to minimize
3204     document.getElementById('outputExpandBackdrop').addEventListener('click', function() {

```



```

3205     document.body.classList.remove('output-expanded');
3206     document.getElementById('expandOutputBtn').textContent = '■ Expand';
3207     document.getElementById('expandOutputBtn').title = 'Expand output panel';
3208     document.body.style.overflow = '';
3209 });
3210 // Esc key to minimize
3211 document.addEventListener('keydown', function(e) {
3212     if (e.key === 'Escape' && document.body.classList.contains('output-expanded')) {
3213         document.body.classList.remove('output-expanded');
3214         document.getElementById('expandOutputBtn').textContent = '■ Expand';
3215         document.getElementById('expandOutputBtn').title = 'Expand output panel';
3216         document.body.style.overflow = '';
3217     }
3218 });
3219 }
3220
3221 ■ (function init() {
3222 ■     buildWordInputs();
3223 ■     bindUI();
3224 ■     try {
3225 ■         const saved = localStorage.getItem('arculusTheme') || (localStorage.getItem('arculusDarkMode') ===
3226 ■             applyTheme(saved);
3227 ■     } catch {
3228 ■         applyTheme('light');
3229 ■     }
3230 ■     refreshWordEnabled();
3231 ■     refreshNetworkStatus();
3232 styleMnemonicBox();
3233 uiStatus('Ready');
3234 startIdleWatcher();
3235 // Ensure derivation info icon reflects default state after full init
3236 const _icon = document.getElementById('derivationInfoIcon');
3237 if (_icon) _icon.style.display = (document.getElementById('derivation')?.value?.trim() === "m/0") ? '
3238 ');
3239
3240 // ████████████████████████████████████████████████████████████████████████████████████████████████
3241
3242 // Self-contained QR Code generator (no external dependencies)
3243 // Supports alphanumeric + byte mode; ECC levels M/Q; versions 1-10
3244 // ████████████████████████████████████████████████████████████████████████████████████████████████
3245 const QRGen = (() => {
3246     // GF(256) arithmetic for Reed-Solomon
3247     const EXP = new Uint8Array(512), LOG = new Uint8Array(256);
3248     (function buildGF() {
3249         let x = 1;
3250         for (let i = 0; i < 255; i++) {
3251             EXP[i] = x; LOG[x] = i;
3252             x = x << 1; if (x > 255) x ^= 0x11d;
3253         }
3254         for (let i = 255; i < 512; i++) EXP[i] = EXP[i - 255];
3255     })();
3256     const gfMul = (a, b) => a && b ? EXP[LOG[a] + LOG[b]] : 0;
3257     const gfPoly = ec => {
3258         let p = [1];
3259         for (let i = 0; i < ec; i++) {
3260             const q = [1, EXP[i]];
3261             const r = new Uint8Array(p.length + 1);
3262             for (let a = 0; a < p.length; a++) for (let b = 0; b < q.length; b++) r[a+b] ^= gfMul(p[a], q[b]);
3263             p = Array.from(r);
3264         }
3265         return p;
3266     };
3267     const rsEncode = (data, ec) => {

```

```

3267         const gen = gfPoly(ec), out = new Uint8Array(data.length + ec).fill(0);
3268         for (let i = 0; i < data.length; i++) out[i] = data[i];
3269         for (let i = 0; i < data.length; i++) {
3270             const c = out[i];
3271             if (c) for (let j = 0; j < gen.length; j++) out[i+j] ^= gfMul(gen[j], c);
3272         }
3273         return out.slice(data.length);
3274     };
3275
3276     // Version capacity tables [version][ecLevel] = {dataBytes, ecPerBlock, blocks}
3277     const CAP = [
3278         null,
3279         [{db:16,ec:10,b:1},{db:13,ec:13,b:1}],
3280         [{db:28,ec:16,b:1},{db:22,ec:22,b:1}],
3281         [{db:44,ec:26,b:2},{db:32,ec:18,b:2}],
3282         [{db:64,ec:18,b:2},{db:48,ec:26,b:2}],
3283         [{db:86,ec:24,b:2},{db:64,ec:18,b:4}],
3284         [{db:108,ec:16,b:4},{db:84,ec:24,b:4}],
3285         [{db:124,ec:18,b:4},{db:93,ec:18,b:6}],
3286         [{db:154,ec:22,b:4},{db:122,ec:22,b:6}],
3287         [{db:182,ec:22,b:5},{db:154,ec:20,b:7}],
3288         [{db:216,ec:26,b:5},{db:180,ec:24,b:8}],
3289     ];
3290
3291     // Format info strings (mask 0-7)
3292     const FMT_M = [0b101010000010010,0b101000100100101,0b101111001111100,0b101101101001011,
3293         0b100010111111001,0b100000011001110,0b100111110010111,0b100101010100000];
3294     const FMT_Q = [0b011010101011111,0b011000001101000,0b011111100110001,0b011101000000110,
3295         0b010010010110100,0b010000110000011,0b010111011011010,0b010101111101101];
3296
3297     // Alignment pattern positions per version
3298     const ALIGN = [null,[],[6,18],[6,22],[6,26],[6,30],[6,34],[6,22,38],[6,24,42],[6,26,46],[6,28,50]]
3299
3300     function makeMatrix(ver) {
3301         const size = ver * 4 + 17;
3302         const m = Array.from({length:size},()=>new Array(size).fill(null));
3303
3304         // Structure patterns use 1 (dark) and 0 (light).
3305         // This stops applyMask() from flipping them, since it only XORs booleans.
3306         const set = (r,c,v) => { if (r>=0&&r<size&&c>=0&&c<size) m[r][c] = v ? 1 : 0; };
3307
3308         const finder = (tr,tc) => {
3309             for (let r=-1;r<=7;r++) for (let c=-1;c<=7;c++) set(tr+r,tc+c, r>=0&&r<=6&&c>=0&&c<=6 ? (r===0
3310         );
3311         finder(0,0); finder(0,size-7); finder(size-7,0);
3312         for (let i=8;i<size-8;i++) { set(6,i,(i&1)===0); set(i,6,(i&1)===0); }
3313         set(size-8,8,true);
3314         const ap = ALIGN[ver];
3315         for (const r of ap) for (const c of ap) {
3316             if (m[r][c]!==null) continue;
3317             for (let dr=-2;dr<=2;dr++) for (let dc=-2;dc<=2;dc++)
3318                 set(r+dr,c+dc, dr===-2||dr===2||dc===-2||dc===2||(dr===0&&dc===0));
3319         }
3320         for (let i=0;i<9;i++) { if(m[8][i]===null) set(8,i,false); if(m[i][8]===null) set(i,8,false); }
3321         for (let i=0;i<8;i++) { set(8,size-1-i,false); set(size-1-i,8,false); }
3322         return m;
3323     }
3324
3325     function placeData(m, data) {
3326         const size = m.length;
3327         let bit = 0, up = true;
3328         for (let col = size-1; col >= 1; col -= 2) {

```

```

3329         if (col === 6) col--;
3330     for (let row2 = 0; row2 < size; row2++) {
3331         const row = up ? size-1-row2 : row2;
3332         for (let dx = 0; dx < 2; dx++) {
3333             const c = col - dx;
3334             if (m[row][c] === null) {
3335                 // Data modules are strictly written as booleans
3336                 m[row][c] = bit < data.length * 8 ? ((data[bit>>3] >> (7-(bit&7))) & 1) === 1 : false;
3337                 bit++;
3338             }
3339         }
3340     }
3341     up = !up;
3342 }
3343 }
3344
3345 function applyMask(m, mask) {
3346     const size = m.length;
3347     const fns = [
3348         (r,c)=>(r+c)%2===0,
3349         (r,c)=>r%2===0,
3350         (r,c)=>c%3===0,
3351         (r,c)=>(r+c)%3===0,
3352         (r,c)=>(Math.floor(r/2)+Math.floor(c/3))%2===0,
3353         (r,c)=>(r*c)%2+(r*c)%3===0,
3354         (r,c)=>((r*c)%2+(r*c)%3)%2===0,
3355         (r,c)=>((r+c)%2+(r*c)%3)%2===0,
3356     ];
3357     const fn = fns[mask];
3358     const out = m.map(row=>row.slice());
3359     for (let r=0;r<size;r++) for (let c=0;c<size;c++) {
3360         // By checking 'boolean', we only apply the mask to data/EC bits.
3361
3362         if (typeof out[r][c] === 'boolean') {
3363             out[r][c] = out[r][c] !== fn(r,c);
3364         }
3365     }
3366     return out;
3367 }
3368
3369 function writeFormat(m, ecLevel, mask) {
3370     const fmt = ecLevel === 1 ? FMT_Q[mask] : FMT_M[mask];
3371     const size = m.length;
3372     const bits = i => (fmt >> (14-i)) & 1 ? 1 : 0;
3373     const set = (r,c,v) => { if (r>=0&&r<size&&c>=0&&c<size) m[r][c] = v; };
3374
3375     // Bits 0-5
3376     for(let i=0; i<6; i++) { set(8, i, bits(i)); set(size - 1 - i, 8, bits(i)); }
3377     // Bit 6
3378     set(8, 7, bits(6)); set(size - 7, 8, bits(6));
3379     // Bit 7
3380     set(8, 8, bits(7)); set(8, size - 8, bits(7));
3381     // Bit 8
3382     set(7, 8, bits(8)); set(8, size - 7, bits(8));
3383     // Bits 9-14
3384     for(let i=9; i<15; i++) { set(14 - i, 8, bits(i)); set(8, size - 15 + i, bits(i)); }
3385 }
3386
3387 function penaltyScore(m) {
3388     const size = m.length;
3389     let score = 0;
3390     // Convert to strict boolean to ensure 1 and true compare properly during the run check
3391     const val = (r,c) => !!m[r][c];

```

```

3391     for (let r=0;r<size;r++) {
3392         let run=1;
3393         for (let c=1;c<size;c++) { if(val(r,c)===val(r,c-1)){run++;}else{if(run>=5)score+=run-2;run=1;
3394             if(run>=5)score+=run-2;
3395         }
3396     for (let c=0;c<size;c++) {
3397         let run=1;
3398         for (let r=1;r<size;r++) { if(val(r,c)===val(r-1,c)){run++;}else{if(run>=5)score+=run-2;run=1;
3399             if(run>=5)score+=run-2;
3400         }
3401     for (let r=0;r<size-1;r++) for (let c=0;c<size-1;c++)
3402         if(val(r,c)===val(r,c+1)&&val(r,c)===val(r+1,c)&&val(r,c)===val(r+1,c+1)) score+=3;
3403     return score;
3404 }
3405
3406 function buildCodewords(bits, totalData, ecPerBlock, numBlocks) {
3407     bits.push(0,0,0,0);
3408     while (bits.length % 8) bits.push(0);
3409     const padBytes = [0b11101100, 0b00010001];
3410     let pi = 0;
3411     while (bits.length < totalData * 8) { for (let b=7;b>=0;b--) bits.push((padBytes[pi]>>b)&1); pi=
3412     const data = new Uint8Array(totalData);
3413     for (let i=0;i<totalData;i++) for (let b=0;b<8;b++) data[i]=(data[i]<<1)|bits[i*8+b];
3414     const blockSize = Math.floor(totalData / numBlocks);
3415     const longBlocks = totalData % numBlocks;
3416     const blocks = [], ecBlocks = [];
3417     let offset = 0;
3418     for (let b=0;b<numBlocks;b++) {
3419         const len = b < numBlocks - longBlocks ? blockSize : blockSize + 1;
3420         const block = data.slice(offset, offset+len); offset+=len;
3421
3422         blocks.push(block); ecBlocks.push(rsEncode(block, ecPerBlock));
3423     }
3424     const out = [];
3425     const maxLen = Math.max(...blocks.map(b=>b.length));
3426     for (let i=0;i<maxLen;i++) for (const b of blocks) if(i<b.length) out.push(b[i]);
3427     for (let i=0;i<ecPerBlock;i++) for (const b of ecBlocks) out.push(b[i]);
3428     return new Uint8Array(out);
3429 }
3430
3431 function generate(text) {
3432     const enc = new TextEncoder().encode(text);
3433     const byteLen = enc.length;
3434     let ver = 1, ecLevel = 0;
3435
3436     // Exact capacity check dynamically factoring in version indicator lengths
3437     for (let v=1;v<=10;v++) {
3438         let reqBytes = Math.ceil((4 + (v < 10 ? 8 : 16) + byteLen * 8) / 8);
3439         if (CAP[v][0].db >= reqBytes) { ver=v; ecLevel=0; break; }
3440         if (v===10 && CAP[v][1].db >= reqBytes) { ver=v; ecLevel=1; break; }
3441         if (CAP[v][1].db >= reqBytes) { ver=v; ecLevel=1; break; }
3442     }
3443
3444     const bits = [0,1,0,0];
3445     const lenBits = ver < 10 ? 8 : 16;
3446     for (let i=lenBits-1;i>=0;i--) bits.push((byteLen>>i)&1);
3447     for (const b of enc) for (let i=7;i>=0;i--) bits.push((b>>i)&1);
3448
3449     const cap = CAP[ver][ecLevel];
3450     const codewords = buildCodewords(bits, cap.db, cap.ec, cap.b);
3451     const base = makeMatrix(ver);
3452     placeData(base, codewords);

```

[illegible]

```

3515     const matrix = QRGen.generate(uri);
3516     QRGen.render(matrix, canvas, { size: 256, quiet: 4, dark: '#000000', light: '#ffffff' });
3517     canvas.style.display = '';
3518     label.textContent = uri;
3519     label.style.display = '';
3520     saveRow.style.display = '';
3521   } catch(e) {
3522     canvas.style.display = 'none';
3523     label.textContent = 'Error: ' + e.message;
3524     label.style.display = '';
3525     saveRow.style.display = 'none';
3526   }
3527 }
3528
3529 function resetQrOutput() {
3530   document.getElementById('qrCanvas').style.display = 'none';
3531   document.getElementById('qrAddressLabel').style.display = 'none';
3532   document.getElementById('qrSaveRow').style.display = 'none';
3533   document.getElementById('qrMemoPrompt').style.display = 'none';
3534 }
3535
3536 function openQrModal() {
3537   resetQrOutput();
3538   const input = document.getElementById('qrAddressInput');
3539   const memoInput = document.getElementById('qrMemoInput');
3540   input.value = '';
3541   memoInput.value = '';
3542   const dlg = document.getElementById('qrModal');
3543   if (typeof dlg.showModal === 'function') dlg.showModal(); else dlg.setAttribute('open', '');
3544   setTimeout(() => input.focus(), 50);
3545 }
3546
3547 function bindQrUI() {
3548   document.getElementById('qrAddressesBtn').addEventListener('click', openQrModal);
3549   document.getElementById('qrModalCloseBtn').addEventListener('click', () => document.getElementById('qrModal').close());
3550   document.getElementById('qrModal').addEventListener('click', e => {
3551     if (e.target === document.getElementById('qrModal')) document.getElementById('qrModal').close();
3552   });
3553
3554   // Main Generate button – for XRP: show memo prompt; for all others: render immediately.
3555   function handleGenerate() {
3556     const address = document.getElementById('qrAddressInput').value.trim();
3557     if (!address) return;
3558     resetQrOutput();
3559     if (isXrpAddress(address)) {
3560       document.getElementById('qrMemoInput').value = '';
3561       document.getElementById('qrMemoPrompt').style.display = '';
3562       setTimeout(() => document.getElementById('qrMemoInput').focus(), 50);
3563     } else {
3564       renderQrFromAddress(address, '');
3565     }
3566   }
3567
3568   document.getElementById('qrGenerateBtn').addEventListener('click', handleGenerate);
3569   document.getElementById('qrAddressInput').addEventListener('keydown', e => {
3570     if (e.key === 'Enter') handleGenerate();
3571   });
3572
3573   // XRP memo prompt – confirm (with tag) or skip (without tag)
3574   document.getElementById('qrMemoConfirmBtn').addEventListener('click', () => {
3575     const address = document.getElementById('qrAddressInput').value.trim();
3576     const memo = document.getElementById('qrMemoInput').value.trim();

```

```

3577     document.getElementById('qrMemoPrompt').style.display = 'none';
3578     renderQrFromAddress(address, memo);
3579 });
3580 document.getElementById('qrMemoSkipBtn').addEventListener('click', () => {
3581     const address = document.getElementById('qrAddressInput').value.trim();
3582     document.getElementById('qrMemoPrompt').style.display = 'none';
3583     renderQrFromAddress(address, '');
3584 });
3585 document.getElementById('qrMemoInput').addEventListener('keydown', e => {
3586     if (e.key === 'Enter') document.getElementById('qrMemoConfirmBtn').click();
3587 });
3588
3589 document.getElementById('qrSaveBtn').addEventListener('click', () => {
3590     const canvas = document.getElementById('qrCanvas');
3591     const address = document.getElementById('qrAddressInput').value.trim();
3592     const safeName = address.replace(/[^a-zA-Z0-9]/g, '_').slice(0, 40) || 'address';
3593     const a = document.createElement('a');
3594     a.href = canvas.toDataURL('image/png');
3595     a.download = `qr_${safeName}.png`;
3596     a.click();
3597 });
3598 document.getElementById('qrCopyAddrBtn').addEventListener('click', async () => {
3599     const address = document.getElementById('qrAddressInput').value.trim();
3600     if (!address) return;
3601
3602     await copyTextToClipboard(address);
3603     const btn = document.getElementById('qrCopyAddrBtn');
3604     const orig = btn.textContent;
3605     btn.textContent = 'Copied!';
3606     setTimeout(() => btn.textContent = orig, 1500);
3607 });
3608
3609 // Edit – collapse back to the address input step, keeping the current value,
3610 // so the user can adjust the address or memo without reopening the modal.
3611 document.getElementById('qrEditBtn').addEventListener('click', () => {
3612     resetQrOutput();
3613     const address = document.getElementById('qrAddressInput').value.trim();
3614     if (isXrpAddress(address)) {
3615         document.getElementById('qrMemoPrompt').style.display = '';
3616         setTimeout(() => document.getElementById('qrMemoInput').focus(), 50);
3617     } else {
3618         setTimeout(() => document.getElementById('qrAddressInput').focus(), 50);
3619     }
3620 });
3621
3622 bindQrUI();
3623 </script>
3624 </body>
3625 </html>
3626

```